

Log to Incident

Gary Brooks

2026-07-28

Table of Contents

1. Purpose	1
2. Log to Incident	2
2.1. The Process	2
2.2. Stage 1: Application Logging	2
2.3. Stage 2: Observability Tooling	2
2.4. Stage 3: ITSM Platform	3
2.5. Client and server applications	4
2.6. Platform implementations	4
3. Spark Architecture	5
3.1. Cluster Mode	5
3.2. Client Mode	6
4. Kubernetes Architecture	8
4.1. Purpose of Kubernetes	8
4.2. Desired state and the object hierarchy	8
4.3. Lifetime bands	10
4.4. Kubernetes objects Spark uses	11
4.4.1. Cluster and nodes	12
4.4.2. Namespace	12
4.4.3. Workload controllers: Deployment and StatefulSet	12
4.4.4. ReplicaSet	13
4.4.5. Pod	13
4.4.6. Container and process	13
4.4.7. Service	13
4.4.8. ConfigMap and the rest of the declared furniture	14
4.5. Other Kubernetes objects	14
4.6. The Spark Worker Deployment template	15
4.6.1. Why the same labels appear on the controller and on the template	16
4.7. From Kubernetes objects to Dynatrace to ServiceNow	17
5. Log Event Management	20
5.1. ServiceNow Modeling - Stage 1	21
5.2. Spark Application - Stage 2	21
5.2.1. Application modeling	21
5.2.2. Log file generation	21
5.3. Dynatrace (Stage 3)	21
5.3.1. Log file ingestion	21
5.3.2. Log Routing	22
5.3.3. Event Processing	22

5.3.4. Log Processing	22
5.3.5. Problem Processing	22
5.3.6. Problem Notification	22
5.3.7. Event Storage	22
5.4. ServiceNow (Stage 4)	23
5.4.1. Event Ingestion	23
5.4.2. Event Creation	23
5.4.3. Alert Creation	23
5.4.4. Incident Creation	23
6. Stage 1 — ServiceNow Modeling	24
6.1. Common CSDM Model	24
6.2. Service-Side Model	26
6.3. Client-side Model	28
6.3.1. Kubernetes workload labels	29
6.3.2. Tag-Based Service Mapping and svc_ci_assoc	30
6.3.3. ServiceNow — SGC Topology Import	30
6.3.4. Dynatrace — management zone	31
7. Spark Application - Stage 2	32
7.1. Service Instance Association	32
7.1.1. Cluster Mode	32
7.1.2. Client Mode	32
7.2. Log File Generation	32
7.2.1. Cluster Mode	33
7.2.2. Client Mode	34
8. Dynatrace	36
8.1. Problem Payload	36
8.1.1. Problem Details JSON	37
8.1.2. Problem Severity	38
8.1.3. Impact Tier	41
8.1.4. Problem Details	41
8.1.5. Ranked Events	43
8.1.6. Custom Properties	44
8.1.7. Ranked Impacts	46
8.1.8. Tags Of Affected Entities	46
8.2. ServiceNow Integration	46
8.2.1. Service Instance Tagging	47
8.3. OneAgent	48
8.3.1. Log Metadata	48
8.3.2. Cluster Mode	49

8.3.3. Client Mode	49
8.3.4. Next Steps	50
8.4. Log Routing	50
8.5. OpenPipeline processing	51
8.5.1. Processors	52
8.5.2. Davis	53
8.5.3. Event Storage	56
8.6. Problem Alerting	57
8.6.1. Alerting Profile	57
8.6.2. Problem notification payload template	58
9. Stage 3 — ServiceNow	60
9.1. Event Management	61
9.2. CI binding procedure	62
9.3. Entity type to CMDB class	62
9.4. Alert Management	62
9.5. Description and alert text	62
9.6. SGC and Event Management configuration	63
9.7. Create incident bound to service instance	63
9.8. Severity policy	63
9.9. Layer model	64
9.10. Incident correlation	65
9.10.1. Service-side	65
9.10.2. Client-side	66
9.11. CSDM identifier lookup	66
9.12. Davis event identity vs problem merge	67
9.13. ServiceNow incident automation specification	68
9.14. Business rule order and rationale	68
9.15. Alert to incident linkage (em_alert.incident)	69
9.16. Problem and alert lifecycle (OPEN / RESOLVED)	69
9.17. OpenPipeline and custom-source attributes (generic Log4j pattern)	69
9.18. Event Management rules (manual configuration)	70
9.19. Script Includes	71
9.19.1. ResolveApplicationService	71
9.19.2. EvtMgmtCustomIncidentPopulator	72
9.19.3. L2IIncidentFromAlert	72
9.20. Business rules (prescriptive configuration)	72
9.21. Deployment Summary	73
9.22. Example resolution	73
10. Appendix A - Definitions	75

11. Appendix B - Dynatrace	76
11.1. Severity levels	76
11.2. Event Types	76
11.2.1. Spelling quirks (important)	80
11.2.2. How matching works	80
11.2.3. Not the same as numeric <code>event.severity</code> 1–5	81

Chapter 1. Purpose

Large incident resolution times hurt business operations and customer satisfaction. One way to shorten resolution is to bring critical application problems to the right operations team quickly and with enough context to act. That requires bridging **observability** (detecting and describing problems from runtime signals) with **IT service management** (tracking work as incidents, assignment, and closure).

This document defines that bridge as the **Log to Incident** process: selected application log messages become managed incidents for the application operations team. The process is named for the **signal type** and **outcome**, not for any particular vendor or tool. Implementations may use different observability and ITSM platforms; this document's worked example uses Apache Spark as the application, with platform-specific sections for how those platforms are configured.

Chapter 2. Log to Incident

Log to Incident is the Observability process that detects and turns **critical** application log messages into **tracked** IT incidents so the application operations team can respond in time.

2.1. The Process

The Log to Incident process takes one or more log messages and generates an incident in the ITSM platform through a three phase process:

1. The application logs a message
2. The observability tooling detects the message as being salient and creates an alert and sends it to the ITSM platform
3. The ITSM platform determines which alerts are salient and then creates an incident for operators to track.

[Log to Incident process phases](#) illustrates these phases in more detail.

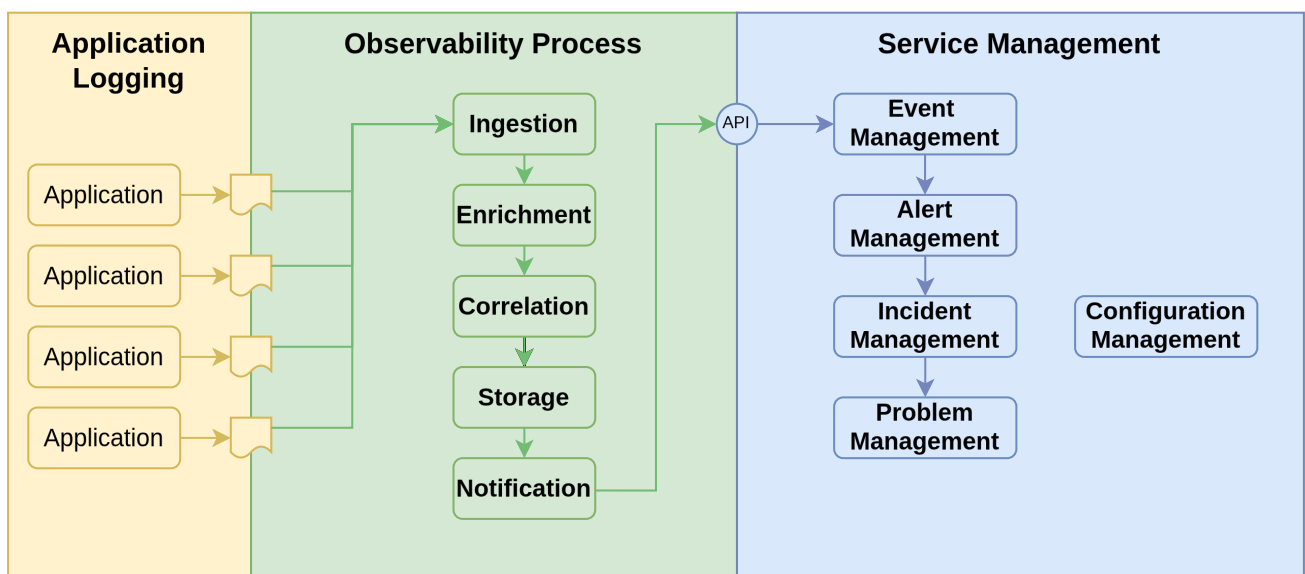


Figure 1. Log to Incident process phases

2.2. Stage 1: Application Logging

To various degrees applications are written to generate log messages to a file or other storage medium or APIs. The log messages can indicate forward progress in the application or problems that have occurred. The problems can range from minor inconsistencies to major failures that require immediate attention.

2.3. Stage 2: Observability Tooling

The observability platform ingests the log messages either through an agent reading the log files or via an API that the application calls. Either way, the observability platform processes each log

message in four addition steps.

1. **Enrichment:** The observability platform adds additional information to the log message. This information comes from the context of the log file (e.g. name of the log file) as well as by parsing information within the log file. This information can include the application name, the environment, the host name, the process id, the thread id, the timestamp, the log level, the log message, and the log message id.
2. **Storage:** The observability platform takes the enriched log message and stores it in a database for fast retrieval and further processing.
3. **Correlation:** The observability platform correlates the log message with other log messages or events to group related log messages into a single message to increase the signal to noise ration and to identify patterns and relationships with other log messages or events.
4. **Notification:** The observability platform then takes selected messages or message groups and sends the enriched log message to the ITSM platform as an event.

Coordination between the application developers and the observability platform engineers. Ideally, this collaboration would begin in the application design phase to ensure that the log messages are structured in a way that that simplifies the enrichment process and identification of critical log messages.

2.4. Stage 3: ITSM Platform

The ITSM platform accepts events from the observability platform and determines which events (aka log messages in this cae) are salient and then creates an incident for operators to track. This happens in a three stage process with a fourth follow on stage to handle multiple incidents that are related to the same underlying problem.

1. **Event Management:** Event management accepts an incoming event (log message) from the observability platform. Events may be clear events that indicate a past issue has been resolved. Or they may be events that indicate a new issue has been detected. Salient events are then forwarded to Alert Management.
2. **Alert Management:** Alert Management receives the salient events from Event Management and creates an alert record in the Alert Management platform. Alerts are bound (mapped) to a configuration item (CI) that best represents the application or the infrastructure component that best represents the component causing or involved in the event. Mapping an alert to a CI requires designing the CMdb model and the information in the event so that Alert Management can use the information sent in the event(s) to map the alert to the correct CI. Multiple events may be mapped to a single alert depending on a message key or other criteria. Alert management will take a clear event will close out an existing, an alert with the same message key. Salient alerts are then forwarded to Incident Management. In **this** implementation, **log** alerts bind the **service instance**; infrastructure alerts (host CPU, Kubernetes workload) bind the HOST or the workload controller.
3. **Incident Management:** Incident Management is where operations teams interact with incidents to resolve the root causes of the incident. When incident management receives a salient alert, it will either bind the alert to an existing incident or create a new incident. The

incident will also be bound to a configuration item (CI) that best represents the application or the infrastructure component that best represents the component causing or involved in the event. Mapping an incident to a CI requires designing the CMdb model and the information in the alert so that Incident Management can use the information sent in the alert to map the incident to the correct CI.

Incident Management will also assign (or determine) the support organization that will be responsible for resolving the incident. The incident will be assigned to the application's support organization and will carry enough log context for diagnosis.

4. **Problem Management:** Problem Management focuses on the root cause analysis of recurring incidents (once service has been restored). A problem will typically map onto several closed incidents. This process is outside the scope of this document.

2.5. Client and server applications

Many systems have both **service-side** components (clustered services, pods, long-running servers) and **client-side** components (drivers, desktops, mobile apps, or other unmanaged endpoints). Clients complicate modeling the application in a CMdb as the client-side components are not managed by the CMdb.

- **Service-side:** map from the running workload's identity to the service instance that operations supports.
- **Client-side:** map from a durable log-path or naming contract to a logical service instance that represents the client population.

Log to Incident must still hand both scenarios, even when the client is not a managed CI in the CMDB.

2.6. Platform implementations

Subsequent chapters describe how to implement the Log to Incident process on specific observability and service management platforms. To design and configure such systems, we need to have a specific application in mind that we will use to illustrate the process. For historical reasons, we will use Apache Spark as the application. Apache Spark is a sophisticated data and analytics platform that runs on Kubernetes and has both client and service use cases. Information on Apache Spark is available at [Apache Spark Documentation](#).

Chapter 3. Spark Architecture

Before we discuss how to observe and manage an application, we need to understand the architecture of the application. Spark in this lab is a Kubernetes application with a **Master**, one or more **Workers**, a **History Server**, and a **Driver**. Those four names are easy to collapse; Log to Incident needs them kept distinct.

- **Spark Master** is the standalone **cluster manager**. It is the Kubernetes StatefulSet `spark-master`. Workers register with it; it allocates executors. It is **not** the process that runs your PySpark program.
- **Spark Worker** is a node that runs executors. You can scale Workers horizontally or vertically.
- **Spark History Server** stores completed-application event logs for debugging and monitoring.
- **Spark Driver** is the process that runs the application (`SparkContext` / `SparkSession`). It talks to the Master to obtain executors. The Driver exists in **both** modes; what changes is **where it runs** and **how the user interacts with it**.

Spark can run in **Client Mode** or **Cluster Mode**.

3.1. Cluster Mode

In cluster mode the user submits a program (a file) to the cluster. The Master **schedules a Driver in the cluster** — typically as a Kubernetes pod — and that Driver runs the submitted file as a **batch job**, not an interactive REPL. The Driver then asks the Master for executors on the Workers. Master and Driver are therefore related in cluster mode (the Master launches the Driver), but they are **not the same Kubernetes object**: the Master remains `spark-master`; the Driver is a separate, usually short-lived, application process. [Spark Cluster Mode Architecture](#) illustrates the cluster mode architecture.

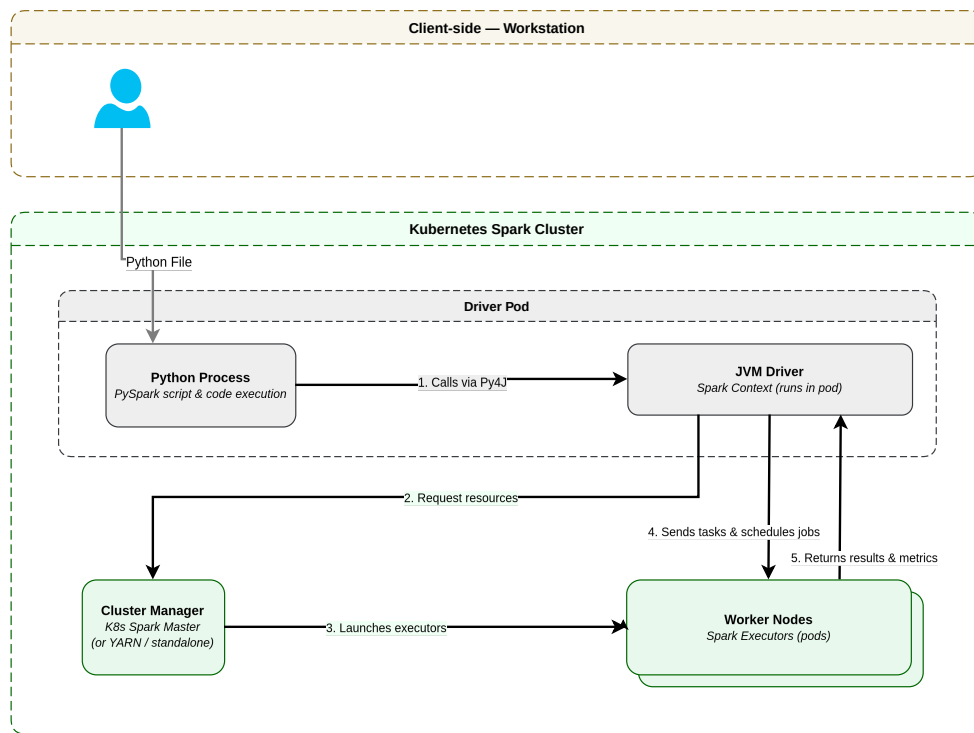


Figure 2. Spark Cluster Mode Architecture

The Master, Workers, and History Server are the **service-side** workloads. Their application logs go to container stdout. Cluster-mode Driver logs, when present, are also in-cluster; this implementation's OpenPipeline maps the durable service-side pod-name prefixes (**spark-master-**, **spark-worker-**, **spark-history-***) to CSDM service instances.

Table 1. Cluster Mode — Log to Incident identity

Aspect	Value
Log path	Container stdout (kubelet <code>/var/log/pods/<namespace>_<pod>_<uid>/...</code> on the node)
Dynatrace entity	Kubernetes workload / pod / HOST (SGO may fan out one problem to several ImpactedEntities)
ServiceNow service instance	Spark-Master / Spark-Worker / Spark-History-Server from the OpenPipeline pod-name prefix → sn-service-instance
Log alert CI	That service instance (enrich overwrites HOST/pod SOS)

3.2. Client Mode

Client mode is a client-server architecture: the Driver runs on a workstation (or a host used as a workstation) as an **interactive** PySpark session. The user iterates on DataFrames, Datasets, and Spark SQL. The Driver still talks to the **same** Master and Workers on the service-side; only the Driver has moved off the cluster. The workstation is typically not a managed CI in the CMDB. [Spark Client Architecture](#) illustrates the client architecture.

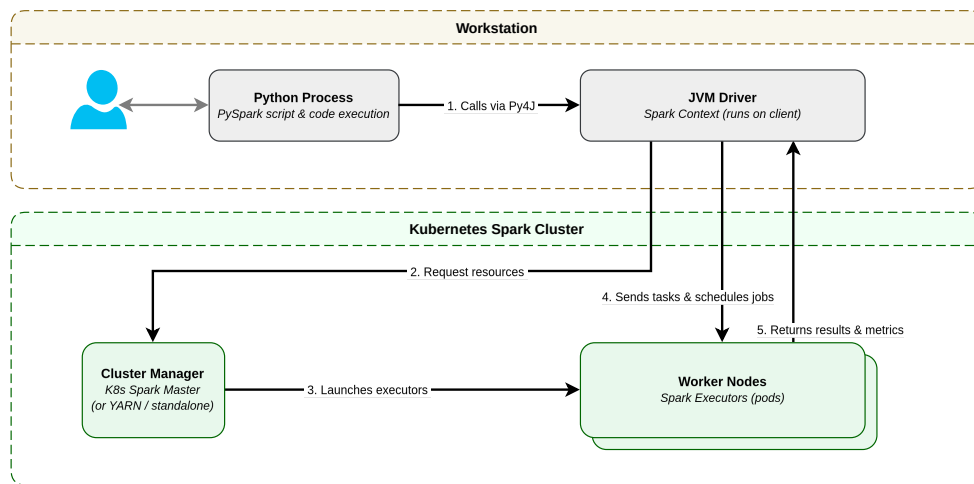


Figure 3. Spark Client Architecture

Table 2. Client Mode — Log to Incident identity

Aspect	Value
Log path	/mnt/spark/client-logs/<SPARK_DRIVER_INSTANCE>/ (node-local; par-a , wrapper PID, ...)
Dynatrace entity	HOST (OneAgent file tail). A CUSTOM_DEVICE “Spark Client” is historical — not the bind.
ServiceNow service instance	Spark-Client from the custom-source service_instance stamp → sn-service-instance
Log alert CI	Spark Client service instance (enrich overwrites HOST SOS)

Spark inherently supports both modes. This reference implementation supports both because Spark does, and because unmanaged clients (desktops, mobile apps) show up in many other applications. The Master/Worker/History Server service-side model is shared; the Driver location is what splits Client from Cluster for logging and CMDB bind.

Chapter 4. Kubernetes Architecture

Kubernetes is the runtime that turns the Spark architecture from the previous chapter into something that actually runs. Spark Master, Spark Worker, and Spark History Server are not processes you start by hand on a host and leave there. They are **workloads** that Kubernetes keeps aligned with a declared desired state: a certain number of Spark Worker processes, a single Spark Master with a stable name, a History Server, and a network identity so clients can find the Master. Dynatrace and ServiceNow both invent a model of that same runtime. Those models only make sense if we are clear about which Kubernetes objects are the long-lived **specification** of Spark, and which objects are the short-lived **instances** Kubernetes creates, replaces, and discards in order to honor that specification.

That distinction is the point of this chapter. Later, in Event Management, we will use it to decide which object an alert or incident should bind to. The short version, which the rest of this chapter prepares: a Spark Worker **pod** is a real running thing, but it is a poor configuration item for service management, because Kubernetes can create and destroy pods faster than ServiceNow can materialize their CMDB records.

4.1. Purpose of Kubernetes

Kubernetes is a cluster operating system for containerized applications. Its job is not to run one container well. Its job is to **continuously reconcile** what operators declared they want with what is actually running on a pool of machines. You do not tell Kubernetes “start these eight Spark Worker processes on lab2.” You tell it “there should be eight Spark Worker replicas, scheduled on lab2, running this image, with these labels and this configuration.” Kubernetes then creates whatever runtime objects it needs to make that statement true, and it keeps creating, replacing, and rescheduling them as nodes fail, images roll, and replica counts change.

That reconciliation loop is why Spark on Kubernetes looks different from Spark on a static set of VMs. The application still has a Master, Workers, and a History Server. What changes is the **unit of replacement**. On a VM, the host and the Spark process tend to live for weeks. On Kubernetes, the **declaration** (the Deployment or StatefulSet) lives for weeks or months; the **process sandbox** that actually executes Spark (the Pod) may live for minutes. Observability and CMDB models that treat every running sandbox as a first-class, durable configuration item inherit that churn.

4.2. Desired state and the object hierarchy

Kubernetes organizes work as a tree of API objects. At the top of the tree are objects you create once and change infrequently: the cluster itself, the nodes that provide CPU and memory, and the namespace that scopes Spark away from other applications. In the middle are **workload controllers** — Deployment, StatefulSet, DaemonSet, CronJob — which hold the desired state and a **pod template**. At the bottom, Kubernetes instantiates ReplicaSets, Jobs, Pods, and Containers from those templates. Pods are scheduled onto nodes. Services select pods by label so the rest of the cluster can reach them without knowing pod IP addresses, which change every time a pod is replaced.

The important modeling idea is that **labels do not inherit automatically down this tree**. A label on a Namespace object is a scope annotation; Kubernetes does not copy it onto Deployments or Pods. A label on a Deployment's own `metadata.labels` (the namespaced object) is likewise **not** copied onto the pods that Deployment creates. What **does** propagate is the pod template inside a controller: `.spec.template.metadata.labels` is copied onto every Pod the controller creates. ReplicaSets created by a Deployment carry the Deployment's selector labels. If ServiceNow or Dynatrace is to see a Spark identity on both the workload CI and the running pods, that identity has to be written in two places in the manifest: on the controller's own `metadata.labels`, and again on the pod template. The Spark manifests in this repository do exactly that with `app.kubernetes.io/` and `servicenow.com/service-instance`. They do **not* put those labels on the Namespace. Why both places — and why not the Namespace — is taken up after the Worker template below.

[Kubernetes object hierarchy for Spark, colored by lifetime](#) is the same tree with Spark names filled in, and with color showing how long each kind of object typically lives. Green objects are the declared desired state. Amber objects are the controller's intermediate revisions. Red objects are the ephemeral runtime.

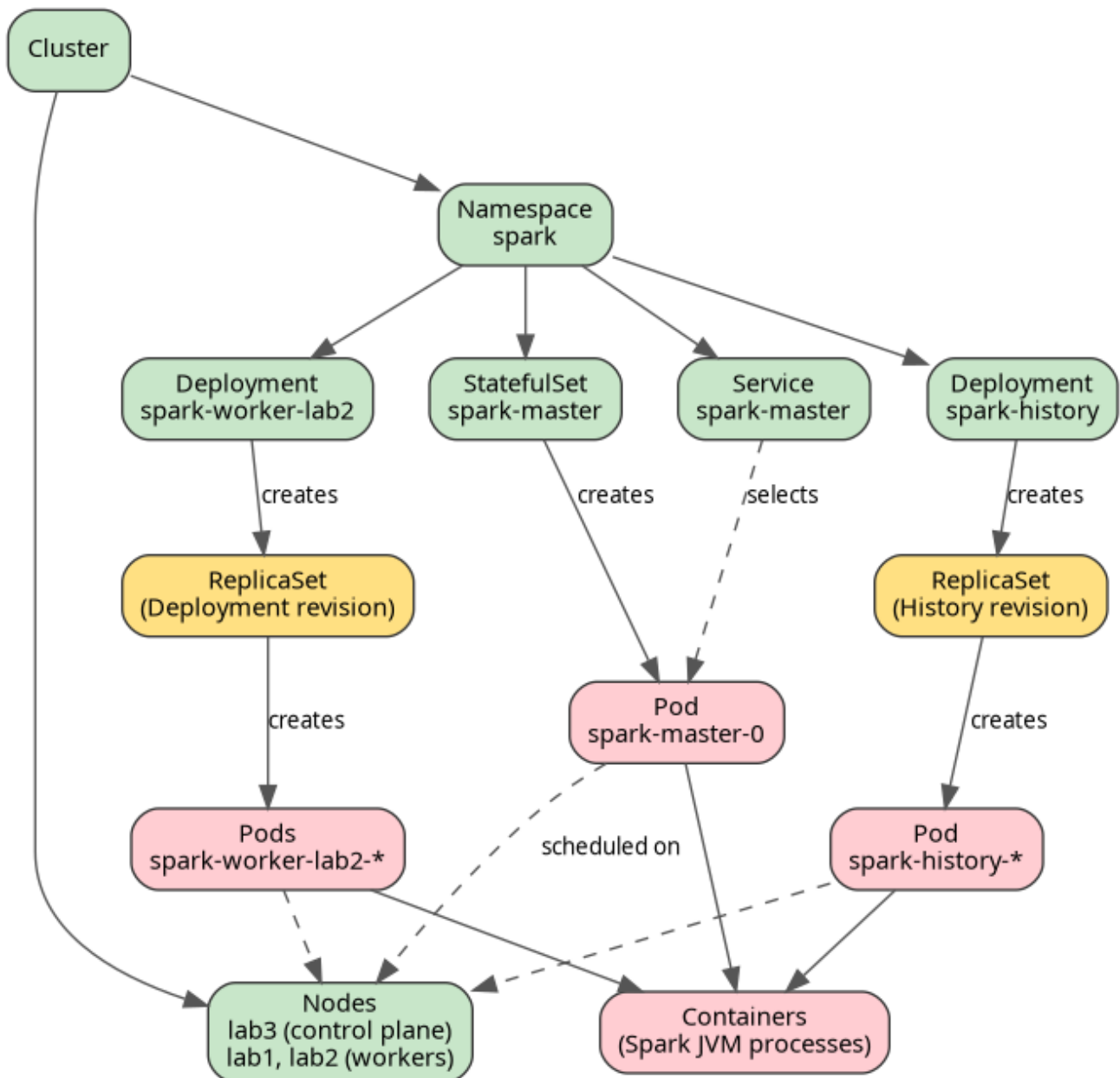


Figure 4. Kubernetes object hierarchy for Spark, colored by lifetime

There **is** an ephemeral level, and it is not a vague property of “the cloud.” It is a specific band of the API tree: **Pod and everything inside it** (containers, and the processes Dynatrace sees inside those containers). Everything above that band is there to **produce** pods. Everything in that band is replaceable. The modeling consequence is that Dynatrace and ServiceNow can usefully represent the green objects as stable entities, must treat the amber objects as implementation detail, and should not depend on the red objects still existing by the time an event has crossed from the cluster into the CMDB.

4.3. Lifetime bands

Kubernetes objects are not equally durable. For Spark — and for almost any application that scales or rolls — three bands are enough.

The **declared band** is the desired state. The cluster, the nodes, the **spark** namespace, the Spark

Master StatefulSet, the Spark Worker Deployments, the History Server Deployment, and the Spark Master Service are created when Spark is installed and they remain until someone changes the install. Replica counts change; the Deployment object itself does not get a new identity. This is the band that should have a CMDB row for the life of the application.

The **revision band** sits between controller and pod. A Deployment never creates pods directly. It creates a ReplicaSet for the current pod-template hash, and that ReplicaSet creates the pods. A rolling update of the Spark Worker image creates a **new** ReplicaSet, shifts replicas across, and garbage collects the old ReplicaSet. A CronJob (which Spark does not use) similarly creates a Job per schedule tick, and the Job creates one or more pods. ReplicaSets and Jobs are real API objects, and ServiceNow Discovery will create CIs for them, but they are not a stable identity for the application. They exist to implement one generation of the template.

The **ephemeral band** is the pod, its containers, and the OS processes inside those containers. A pod is the smallest deployable unit Kubernetes will schedule: one or more containers that share a network namespace, storage volumes, and a unique name. Kubernetes treats pods as disposable. If a Spark Worker container exits, the ReplicaSet creates a replacement pod with a **different** name. If you scale `spark-worker-lab2` from eight replicas to sixteen, eight new pod names appear. If a node is drained, every pod on it is deleted and recreated elsewhere. Pod IP addresses, UIDs, and often names are not durable identifiers. Containers are even shorter-lived: a container restart inside a still-existing pod is a new process in the same sandbox.

Frequency follows from that design. Declared objects are instantiated when Spark is deployed and then updated on a human timescale — hours to months. ReplicaSets are instantiated on each Deployment rollout, so typically days to weeks in a lab, and as often as every pipeline run in a busy environment. Pods are instantiated whenever desired replicas are not met: at deploy time, on every scale event, on every crash, on every OOM kill, on every node preemption, and on every rolling update. In a Spark Worker Deployment of eight replicas, a single image roll replaces eight pods. A crash-looping worker can create and discard pods on a seconds-to-minutes cadence. That is faster than Kubernetes Visibility Agent (KVA) Informer updates and Service Graph Connector imports can reliably create, relate, and tag a `cmdb_ci_kubernetes_pod` row, and it is certainly faster than tag-based Service Mapping can recompute `svc_ci_assoc` membership for each new name.

StatefulSet pods are a mild exception that is easy to over-read. Spark Master is a StatefulSet with `replicas: 1`, so Kubernetes keeps a pod named `spark-master-0`. The **name** is stable. The **pod object** is not: if the Master container dies, Kubernetes deletes that pod and creates a new one that happens to reuse the same name. Dynatrace still sees a new `CLOUD_APPLICATION_INSTANCE`. ServiceNow may merge on name, or it may briefly have a gap. Stability of the name is not the same as immunity to churn.

4.4. Kubernetes objects Spark uses

The Spark install occupies one namespace on one cluster. The sections below walk that tree from the top down, naming each object the way Kubernetes, Dynatrace, and ServiceNow will later name it.

4.4.1. Cluster and nodes

A **cluster** is the control plane plus the member machines. In this lab the control plane runs on lab3 and the Spark executor capacity runs on lab1 and lab2. Dynatrace represents the cluster as `KUBERNETES_CLUSTER` and each member as `KUBERNETES_NODE`. ServiceNow stores those as `cmdb_ci_kubernetes_cluster` and `cmdb_ci_kubernetes_node`. The node is not the Linux host; it is Kubernetes' view of that host as a schedulable worker. The Linux host is a separate object — Dynatrace `HOST`, ServiceNow `cmdb_ci_linux_server` — that OneAgent discovers independently. A pod **runs on** a node; the node **runs on** a host. Both views appear in the Spark service-side CMDB model, and they should not be collapsed: Event Management that binds to the host is binding to the machine that tailed a log file, not to the Spark workload that wrote it.

Nodes are durable relative to pods. They change when you add or remove machines, not when Spark scales. They are the wrong CI for a Spark application incident, because many unrelated pods share a node, but they are a legitimate CI for node pressure, disk, and kubelet problems.

4.4.2. Namespace

A **namespace** is a named scope inside the cluster. Spark's controllers, pods, and Services live in `spark`. Namespaces are how you keep Spark's `spark-master` Service from colliding with some other application's object of the same name. Dynatrace calls this `CLOUD_APPLICATION_NAMESPACE` (also surfaced as a Kubernetes namespace entity). ServiceNow stores it as `cmdb_ci_kubernetes_namespace`.

A namespace is durable and shared. Tagging the namespace and expecting every child to inherit that tag is a ServiceNow Tag Engine convenience, not a Kubernetes behavior. For Spark we tag the workload controllers and their pod templates, not the namespace, so that Spark Worker, Spark Master, and Spark History Server can be three different service instances inside the same namespace.

4.4.3. Workload controllers: Deployment and StatefulSet

A **Deployment** is the usual controller for stateless, horizontally scaled software. It stores a replica count and a pod template. Kubernetes makes the number of matching pods equal the replica count, through ReplicaSets, and it does so without caring about pod names or which node a replica landed on. Spark Workers are Deployments: `spark-worker-lab1` and `spark-worker-lab2`, each with eight replicas pinned to that node by affinity. Spark History Server is also a Deployment, with a single replica. Dynatrace folds every workload controller into one entity type, `CLOUD_APPLICATION`. ServiceNow keeps the Kubernetes kind: `cmdb_ci_kubernetes_deployment` for the Workers and History Server.

A **StatefulSet** is the controller you use when identity matters. Each replica gets a stable ordinal name (`spark-master-0`), a stable network identity through a headless Service, and, if you attach them, stable volumes. Spark Master is a StatefulSet with one replica because clients and Workers find it at a known name and port, and because there is only one Master. Dynatrace still calls this `CLOUD_APPLICATION`. ServiceNow stores it as `cmdb_ci_kubernetes_statefulset`. That class difference matters when Event Management looks up a CI by table: a Master problem must not be required

to land in the Deployment table.

Both controllers are the **specification** of a Spark component. They change when operators change the install — image, replica count, labels, resource limits — not when a JVM crashes. `CLOUD_APPLICATION` / `cmdb_ci_kubernetes_deployment` (or StatefulSet) is therefore the natural durable handle for “this is the Spark Worker workload.”

4.4.4. ReplicaSet

A **ReplicaSet** guarantees that a given pod template is running N times. You almost never create one by hand. The Deployment controller creates a ReplicaSet per template hash and uses it as a generation. ServiceNow Discovery will create `cmdb_ci_kubernetes_replicaset` CIs. Dynatrace does not give ReplicaSets their own first-class entity; they disappear into the parent `CLOUD_APPLICATION`. That is a hint. ReplicaSets are how Deployments **implement** desired state, not how operators **name** Spark. Modeling or binding at the ReplicaSet layer inherits every rollout as a new identity.

4.4.5. Pod

A **Pod** is a running sandbox: one or more containers, a unique UID, a name, an IP, and a node assignment. It is the object `kubectl get pods` lists, the directory name under `/mnt/spark/logs/`, and the thing that disappears when Spark Worker is scaled down. Dynatrace calls each pod `CLOUD_APPLICATION_INSTANCE`. ServiceNow stores it as `cmdb_ci_kubernetes_pod`.

Pods are the ephemeral level. They are also the most tempting CI, because they are what is actually on fire when a container OOMs or a log line is written. The temptation is what later Event Management has to resist. A problem that names a pod that KVA has not yet inserted, or has already deleted, cannot bind in ServiceNow. A problem that names a Deployment can, because the Deployment CI was created when Spark was installed and is still there after the pod is gone.

Spark Master’s `spark-master-0` name looks like a host name. It is a StatefulSet ordinal. Treat it as a convenient label on an otherwise ordinary pod, not as a VM.

4.4.6. Container and process

Inside the pod, Kubernetes runs **containers** from the image in the template. The Spark Worker container executes `start-worker.sh`, which is a JVM. Dynatrace OneAgent, looking from inside the node, also sees a `PROCESS_GROUP` / `PROCESS_GROUP_INSTANCE` for that JVM. ServiceNow may import those as `cmdb_ci_group` and `cmdb_ci_appl`. They are not Kubernetes API objects. They are the process-level view of the same running worker. They churn with the pod: a new pod is a new process instance. They belong in topology. They are not a better bind target than the pod; they are a more granular one.

4.4.7. Service

A Kubernetes **Service** is a stable network identity in front of a changing set of pods. It selects pods by label and gives them a DNS name and a cluster IP (or a NodePort, or a headless DNS

record). Spark Master is reached as `spark-master.spark.svc.cluster.local:7077` because a Service named `spark-master` selects pods labeled `app: spark-master`. There is also a headless Service for the StatefulSet and a NodePort Service for clients outside the cluster. Spark Workers do not need a Service of their own; they register with the Master. Dynatrace represents Services as `KUBERNETES_SERVICE`; ServiceNow as `cmdb_ci_kubernetes_service`.

Services are durable and belong in the declared band. They are how other workloads find Spark, not how Spark runs. An incident about "Workers cannot reach Master" may involve the Service; an incident about a Worker log line does not.

4.4.8. ConfigMap and the rest of the declared furniture

The Worker template also mounts ConfigMaps (`spark-configmap`, `spark-defaults-conf`), uses a ServiceAccount, and attaches hostPath volumes for logs, events, data, and jars. Those objects are part of desired state. They rarely appear as Dynatrace entities and they are not used as Event Management bind targets for Spark logs. They matter because they show that a Deployment is a **bundle**: identity, replica count, pod template, configuration, and storage, not just a container image.

4.5. Other Kubernetes objects

Spark is not the whole cluster. Three other controller kinds show up in Dynatrace and ServiceNow even when Spark never declares them, and they follow the same template-versus-instance pattern.

A **DaemonSet** runs exactly one pod on every node that matches its selector (usually every node). That is the opposite of a Deployment, which runs N replicas wherever the scheduler puts them. Spark does not use a DaemonSet: Master, Worker, and History Server need a chosen replica count and, for the Workers, pinning to specific nodes — not "one Spark process on every machine in the cluster." DaemonSets on this cluster are node-local **agents**. **Dynakube OneAgent** (`dynakube-oneagent` in the `dynatrace` namespace) places a OneAgent pod on every Kubernetes node so Dynatrace can see the host, the processes inside Spark pods, and the log files those pods write. **node-exporter** in the `monitoring` namespace does the same pattern for Prometheus: one exporter per node for CPU, memory, disk, and network metrics. kube-proxy and similar kube-system agents are also DaemonSets. Dynatrace still types the DaemonSet as `CLOUD_APPLICATION` and each per-node pod as `CLOUD_APPLICATION_INSTANCE`. ServiceNow uses `cmdb_ci_kubernetes_daemonset`. The DaemonSet is durable; the per-node pods churn when nodes join or the DaemonSet rolls. Bind to the DaemonSet, not to `node-exporter-xxxx` on lab2.

A **CronJob** is a schedule plus a job template. Each tick creates a **Job**; the Job creates one or more pods, the pods run to completion, and they go away. This is the most aggressive ephemeral pattern Kubernetes offers. A nightly reconciliation that lasts four minutes will produce a Job CI and a pod CI that are already candidates for deletion by the time ServiceNow's next discovery interval runs. Dynatrace types the CronJob as `CLOUD_APPLICATION`. ServiceNow has `cmdb_ci_kubernetes_cron_job` and `cmdb_ci_kubernetes_job`. Spark does not use CronJobs; the lab still has to model them correctly for anything else on the cluster, and they are the cleanest illustration of why instance-level CIs cannot be the service-management identity. Tag the CronJob

controller. Treat Jobs and their pods as runtime exhaust.

Jobs that are not owned by a CronJob — one-shot batch — are the same story with no schedule. They are `CLOUD_APPLICATION` for the Job object if Dynatrace surfaces it, and `cmdb_ci_kubernetes_job` in ServiceNow, with pods underneath that should not be the CMDB identity of the batch **service**.

4.6. The Spark Worker Deployment template

The Spark Worker is the best teaching example because it is a Deployment, it scales, and it is the component whose pods churn. The live template is `ansible/roles/spark/templates/spark-worker.yaml.j2`. The listing below is the lab2 half, reduced to the fields that correspond to the objects in [Kubernetes object hierarchy for Spark, colored by lifetime](#).

Listing 1. Spark Worker Deployment (lab2), annotated against Kubernetes objects

```
apiVersion: apps/v1
kind: Deployment ①
metadata:
  name: spark-worker-lab2 ②
  namespace: spark ③
  labels:
    app: spark-worker
    app.kubernetes.io/name: spark-worker
    app.kubernetes.io/component: worker
    app.kubernetes.io/part-of: apache-spark
    servicenow.com/service-instance: Spark-Worker ④
spec:
  replicas: 8 ⑤
  selector:
    matchLabels:
      app: spark-worker
      node: lab2 ⑥
  template: ⑦
    metadata:
      labels:
        app: spark-worker
        node: lab2
        app.kubernetes.io/name: spark-worker
        servicenow.com/service-instance: Spark-Worker ⑧
    spec:
      affinity:
        nodeAffinity: ⑨
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: kubernetes.io/hostname
                    operator: In
                    values:
                      - lab2
      containers:
        - name: spark-worker ⑩
          image: "{{ spark_image }}:{{ spark_tag }}"
          command: ["/bin/bash", "-c"]
          args:
            - exec /opt/spark/sbin/start-worker.sh spark://spark-master.spark.svc.cluster.local:7077
```

① **Workload controller.** `kind: Deployment` is the declared Spark Worker workload. Dynatrace: `CLOUD_APPLICATION`. ServiceNow: `cmdb_ci_kubernetes_deployment`. This object is created once per worker pool (`spark-worker-lab1`, `spark-worker-lab2`) and kept.

- ② **Controller identity.** `metadata.name` is the stable workload name Dynatrace reports as `k8s.workload.name` and ServiceNow stores as the Deployment CI `name`.
- ③ **Namespace field, not Namespace labels.** `namespace: spark` places this Deployment in the `spark` namespace. It does not label the Namespace object, and it does not copy these labels onto pods.
- ④ **Controller labels** (`metadata.labels` on the Deployment). These labels are on the Deployment object itself — the namespaced resource, sometimes spoken of as the “namespace level” of the manifest to distinguish it from the pod template. KVA writes them to `cmdb_key_value` on `cmdb_ci_kubernetes_deployment`. Kubernetes does **not** copy them onto pods.
- ⑤ **Desired count.** Kubernetes instantiates a ReplicaSet whose job is to keep eight pods running. Changing this number creates or deletes pods; it does not create a new Deployment.
- ⑥ **Selector.** The Deployment owns pods that match these labels. The ReplicaSet uses the same selector. A Service, if we had one, would use a similar selector to find those pods.
- ⑦ **Pod template.** This is not a pod. It is the cookie cutter. Every time the ReplicaSet needs another replica, it copies this template, assigns a generated name (`spark-worker-lab2-
<replicaset-hash>-<random>`), and asks the scheduler to place the new pod on a node.
- ⑧ **Pod-template labels.** Kubernetes copies these onto each created pod. KVA then writes them to `cmdb_key_value` on `cmdb_ci_kubernetes_pod`. The `servicenow.com/service-instance: Spark-Worker` value is the CSDM display name in the Kubernetes label character set (spaces become hyphens). Master and History Server use the same key with `Spark-Master` and `Spark-History-Server`.
- ⑨ **Scheduling onto a Node.** Affinity is how this Deployment stays on lab2. The resulting relationship is Pod **scheduled on** Node, which Dynatrace and ServiceNow both record. Eight Worker pods on lab2 still share one Node CI and one Host CI.
- ⑩ **Container.** The running Spark Worker JVM. Dynatrace also sees this as a process group instance. It dies with the pod.

What the template does **not** name is as important as what it does. There is no pod name. There is no ReplicaSet name. Those are generated at instantiation time, at the frequency of the ephemeral band. The objects you can point to in the manifest months later are the Deployment, the namespace, the Service the Workers **call** (`spark-master.spark.svc.cluster.local`), and the labels that tie this workload to the Spark Worker service instance.

Spark Master, by contrast, is the same story with `kind: StatefulSet` and `replicas: 1`. The template still has controller labels and pod-template labels. The difference is that Kubernetes names the pod `spark-master-0` instead of generating a random suffix. History Server is another Deployment, `replicas: 1`, tagged `Spark-History-Server`. Three controllers, three CSDM service instances, one namespace.

4.6.1. Why the same labels appear on the controller and on the template

The listing puts `app.kubernetes.io/*` and `servicenow.com/service-instance` in two places: on the Deployment’s own `metadata.labels`, and again under `spec.template.metadata.labels`. That is not redundancy Kubernetes needs in order to schedule pods, and it is not a label on the Namespace

object. The `namespace: spark` field only names where the Deployment lives.

Kubernetes never copies a controller's `metadata.labels` onto the pods it creates. Only the pod template's labels are stamped onto each Pod. Discovery follows the same split. KVA creates a `cmdb_ci_kubernetes_deployment` (or StatefulSet) from the controller object and writes **that object's** labels into `cmdb_key_value` on the workload CI. It creates a `cmdb_ci_kubernetes_pod` per running pod and writes the **pod's** labels — which came from the template — onto the pod CI.

Template labels alone would tag only pods. That is the ephemeral band: Service Mapping membership would churn as pods come and go, and Event Management that binds to `CLOUD_APPLICATION / cmdb_ci_kubernetes_deployment` would find a Deployment CI with no `servicenow.com/service-instance` key. Controller labels alone would tag the durable workload CI — which is what we want for service mapping and alert binding — but the running pods would not carry the Spark identity, so anything that still looks at pod CIs (log-path correlation, topology) would miss the tag. Both sets are required because they tag two different CIs.

The `app.kubernetes.io/` labels are the standard Kubernetes application identity (name, instance, component, part-of). Dynatrace and other tooling use them to recognize the workload. The `servicenow.com/service-instance` label is the CSDM join key. We do **not* put `servicenow.com/service-instance: Spark-Worker` on the Namespace, because the namespace is shared by Master, Worker, and History Server, which are three different service instances. A namespace-level tag would collapse them into one map.

4.7. From Kubernetes objects to Dynatrace to ServiceNow

Dynatrace does not copy the Kubernetes API one-for-one. It compresses the declared band into a small set of entity types and it treats the pod as an **instance of** a workload. ServiceNow's Kubernetes CMDB, filled by KVA and by the Service Graph Connector for Dynatrace (SGO-Dynatrace), is closer to the API: Deployment and StatefulSet are different classes, and ReplicaSets exist. The table below is the contract among the three models for the objects this chapter has named. Spark-used objects are listed first; DaemonSet, CronJob, and Job are included because they appear in the same Dynatrace and CMDB taxonomies the moment anything else runs on the cluster.

Table 3. Kubernetes constructs mapped to Dynatrace entities and ServiceNow CMDB classes

Kubernetes object	Spark uses it	Lifetime band	Dynatrace entity	ServiceNow CMDB class
Cluster	Yes	Declared	<code>KUBERNETES_CLUSTER</code>	<code>cmdb_ci_kubernetes_cluster</code>
Node	Yes (lab1, lab2, lab3)	Declared	<code>KUBERNETES_NODE</code>	<code>cmdb_ci_kubernetes_node</code>
Namespace	Yes (<code>spark</code>)	Declared	<code>CLOUD_APPLICATION_NAMESPACE</code>	<code>cmdb_ci_kubernetes_namespace</code>
Deployment	Yes (Workers, History Server)	Declared	<code>CLOUD_APPLICATION</code>	<code>cmdb_ci_kubernetes_deployment</code>

Kubernetes object	Spark uses it	Lifetime band	Dynatrace entity	ServiceNow CMDB class
StatefulSet	Yes (Master)	Declared	CLOUD_APPLICATION	cmdb_ci_kubernetes_statefulset
DaemonSet	No (OneAgent, node-exporter)	Declared	CLOUD_APPLICATION	cmdb_ci_kubernetes_daemonset
CronJob	No	Declared	CLOUD_APPLICATION	cmdb_ci_kubernetes_cronjob
Service	Yes (Master)	Declared	KUBERNETES_SERVICE	cmdb_ci_kubernetes_service
ReplicaSet	Yes (implicit under Worker and History Deployments)	Revision	(none; folded into CLOUD_APPLICATION)	cmdb_ci_kubernetes_replicaset
Job	No	Revision	CLOUD_APPLICATION (when surfaced)	cmdb_ci_kubernetes_job
Pod	Yes	Ephemeral	CLOUD_APPLICATION_INSTANCE	cmdb_ci_kubernetes_pod
Container / process	Yes (Spark JVM)	Ephemeral	PROCESS_GROUP / PROCESS_GROUP_INSTANCE (and container entities)	cmdb_ci_group / cmdb_ci_appl (and container CIs)
Linux host (not a K8s object)	Yes (nodes)	Declared	HOST	cmdb_ci_linux_server

Read the Dynatrace column as a **spec versus instance** model. Every Spark workload controller — Worker Deployment, Master StatefulSet, History Deployment, and, elsewhere, a DaemonSet or CronJob — is one **CLOUD_APPLICATION**. Every pod those controllers create is a **CLOUD_APPLICATION_INSTANCE** of that application. Davis problems about a workload therefore carry **dt.entity.cloud_application**. Problems about a specific sandbox carry **dt.entity.cloud_application_instance**. The first identity survives a crash. The second is the crash.

ServiceNow then has two imports of the same world. KVA watches the Kubernetes API and creates the **cmdb_ci_kubernetes_*** rows, including ReplicaSets that Dynatrace never named. SGO-Dynatrace imports Smartscape entities and IRE-merges them onto those rows when names match, storing the Dynatrace entity id in **sys_object_source**. The merge is why a **CLOUD_APPLICATION** for **spark-worker-lab2** and a KVA Deployment named **spark-worker-lab2** become one CI. It is also why a pod that existed only for forty seconds may appear in Dynatrace, never make it through SGC, and already be gone from the API when KVA next syncs.

Above the Kubernetes classes sits the CSDM model from Stage 1. Apache Spark is the business service. Spark Worker, Spark Master, and Spark History Server are service instances. They are not Kubernetes objects. They are joined to Kubernetes CIs by tag-based Service Mapping on servicenow.com/service-instance, which reads **cmdb_key_value** and records membership in **svc_ci_assoc**. If that tag is on the Deployment and the StatefulSet, the service instance is tied to the declared band. If it is only on pods, the service map's membership churns with the ephemeral band: rows appear and vanish as fast as pods do, and an incident that has to walk **svc_ci_assoc**

from a pod CI will fail whenever the pod CI is missing.

That is the story this chapter is for. Kubernetes runs Spark by holding a durable specification and instantiating disposable pods from it. Dynatrace already distinguishes those two layers as `CLOUD_APPLICATION` and `CLOUD_APPLICATION_INSTANCE`. ServiceNow can represent both, but the CMDB and Event Management cannot keep up with the instance layer. The next time we talk about binding an event to a CI, the candidate is the workload in the declared band — the Spark Worker Deployment, the Spark Master StatefulSet — not the pod that happened to emit the log line.

Chapter 5. Log Event Management

This chapter documents the high-level architecture of the Log to Incident process using **Dynatrace** as the observability platform and **ServiceNow** as the ITSM & Event Management platform. To flesh out the reference architecture we use Apache Spark as the subject application being observed and managed. [Log to Incident data flow](#), below, illustrates the end-to-end flow. Log messages from the Spark application flow through Dynatrace to ServiceNow. This is a three phase process that starts with Spark writing to an application log file; Dynatrace then ingests and processes a log line, deciding which ones are critical issues (Problems) and then notifies ServiceNow of critical issue. ServiceNow ingest the issue (event) and creates an alert and incident mapping bot to appropriate CIs. ServiceNow then notifies the support team of the CI assigned to the incident.

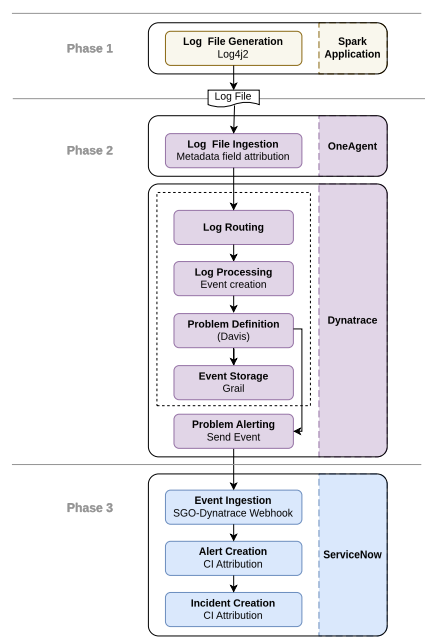


Figure 5. Log to Incident data flow

Critical to event management in ServiceNow is the CMdb, especially the services defined in the CSDM. The definition of services in the CSDM and discovery of the CI model in the CMdb guides the event and incident management processes. Properly setting up the CMdb is required prior to the first log record being written and constitutes a phase unto itself.

In ServiceNow, operational issues are managed at the incident level. The assignment group of the CI assigned to an alert or incident is the group that will be notified when an issue arises. This makes assigning a CI to an incident critical to the process. For application log messages, the assigned CI should be the service instance of the application instance emitting the log message. Finding the right CI at runtime to assign to a log message does not happen automatically. It needs to be designed into the CMdb model of the application.

Designing the CMdb model for the application is another phase in the process. It needs to be done before the application ever runs. The table below enumerates all the major steps required to configure the Log to Incident flow and touches on some of the key design issues for the step.

5.1. ServiceNow Modeling - Stage 1

We need to understand the relationships in the CMdb model and how they relate to runtime components so that at the time a log message (event) is ingested by ServiceNow, we can eventually assign the correct CI to the incident. This in turns understanding the runtime CIs define by Dynatrace and how they relate to the rest of the CMdb model.

5.2. Spark Application - Stage 2

Application teams need to work with the Observability on how to model their application and where the log files are emitted so they can be ingested by Dynatrace.

5.2.1. Application modeling

Application teams must model their applications. Much of the modeling is done in ServiceNow. This can be done either by the application team, Enterprise Architecture team, or the CMdb/CSDM team. For container-based applications, application teams will need to tag their containers in the applications deployment descriptors.

- What tags need to be set on the containers?
- What do those tags map to in the CMdb model?

5.2.2. Log file generation

The application team and the Observability team must work together to decide where and how the application generates the log files.

- Where are the logs stored for which applications?
- How does the application team configure the application to emit the logs to the desired location?

5.3. Dynatrace (Stage 3)

Dynatrace's OneAgent utility, reads in the log files and sends them to Dynatrace (proper) to processes them.

5.3.1. Log file ingestion

Once the teams determine where the log files are emitted, Dynatrace's OneAgent will ingest the log data and send them to Dynatrace (proper) to process them.

- How does Dynatrace's OneAgent know where the logs are?
- What metadata is derived from the log file locations?
- Where does OneAgent then send the log data?

5.3.2. Log Routing

We will need to specify how Dynatrace will route the log messages to the appropriate OpenPipeline which will provide the bulk of the processing.

5.3.3. Event Processing

Each log entry is viewed as an event. The `event.name`, `event.type`, and `dt.source_entity` are important attributes that decide if two events are the same. We must decide how these attributes are set for de-duplication.

5.3.4. Log Processing

Log processing manipulates the log messages (Dynatrace Events) and marshals the data to store in Grail and then send to ServiceNow. .

- What data and meta-data we need to extract from an event (log message)?
- What metadata is needed to store in Grail?
- What metadata is needed to send to ServiceNow?

5.3.5. Problem Processing

An event can be promoted to a problem and then sent to ServiceNow. This gives rise to the following questions:

- What is the criticality of the event and should we promote it to a problem?
- If we promote the event to a problem, do we create a new problem or add it to an existing problem?
- What metadata do we need to associate to the problem and send to ServiceNow?
- How will ServiceNow prioritize the ServiceNow event and alert corresponding to the problem?

5.3.6. Problem Notification

Once we have established marshaled the log messages that are critical we need to specify how Dynatrace sends a Problem for the log message to ServiceNow.

5.3.7. Event Storage

Once any problems are created, Dynatrace will optionally store the events in the Grail database.

- What events should be stored in Grail?

5.4. ServiceNow (Stage 4)

As Dynatrace sends problems to ServiceNow, we transition from observability to event and incident management. In ServiceNow, Dynatrace problems are mapped to ServiceNow events and alerts.

5.4.1. Event Ingestion

The Dynatrace Service Graph Connector will define how ServiceNow defines the endpoint to receive events.

5.4.2. Event Creation

Once an event is ingested we need to specify how ServiceNow creates the event record. This brings to the fore a number of questions: Should we reuse an existing event record (i.e. deduplication) or create a new one? What data and metadata does it need to create the event record? What should we use as the message key?

5.4.3. Alert Creation

Once an event is created, we need to associate it with an alert. How and when does it create an alert for the event and what CI does it assign to the alert?

5.4.4. Incident Creation

When an incident is created, how does ServiceNow identify the correct CI to bind to the incident?

Client and Cluster Mode configuration is [Client and Cluster Mode Use Cases](#). ServiceNow Event Management remains [Stage 3](#).

Chapter 6. Stage 1 — ServiceNow Modeling

The first Phase in the process is to model the application in ServiceNow. It needs to happen even before the application ever runs. The ServiceNow modeling is the data foundation of the Log to Incident process. It consists of the CSDM hierarchy and the pattern-specific CMDB components. We need to do this for the Service-side and the Client-side components.

Before we can bind incidents in ServiceNow to a an appropriate CI, we must model the application in ServiceNow, including both the client-side and service-side use cases. Furthermore, we need to model the application in a manner that follows ServiceNow's CSDM best practices. This involves defining the CSDM hierarchy and discovering pattern-specific CMDB components. We need to do this for the Service-side and the Client-side Spark use cases.

In these models, some CIs (of components) are automatically discovered by ServiceNow and other, typically in the CSDM model, are manually defined to represent the application within an enterprise. Other CIs are imported from other systems, like Dynatrace.

We first examine the CSDM model that is common to both the client and service-side applications. In the client-side scenario we have both a client and service-side application. As such we examine the service-side model next and then the client-side model after that.

6.1. Common CSDM Model

The CSDM is part of the overall ServiceNow model for an application. It models how an application fits within the enterprise. It is a hierarchy of CIs that starts with the service instance, which models the application instance. Associated with the service instance at a lower (conceptual) level are the CIs that represent the application's Kubernetes components. The Business Application (cmdb_ci_service) represents the business' view of the application - i.e. what more generalized, business oriented, service does it provide. In this case, it is an Apache Spark service that data scientists and engineers use to perform data analytics and machine learning. There could be other instances of the Spark Master service in the enterprise. Lastly, the Business Application represents the overall functional capability area. In this case, it is Data and Analytics. There could be other technologies used in the enterprise aside from Apache Spark (e.g. Snowflake), which would get their own Business Service and still fall under the Data and Analytics Business Application.

[Spark CSDM Model](#) illustrates the common CSDM model for the Spark application.

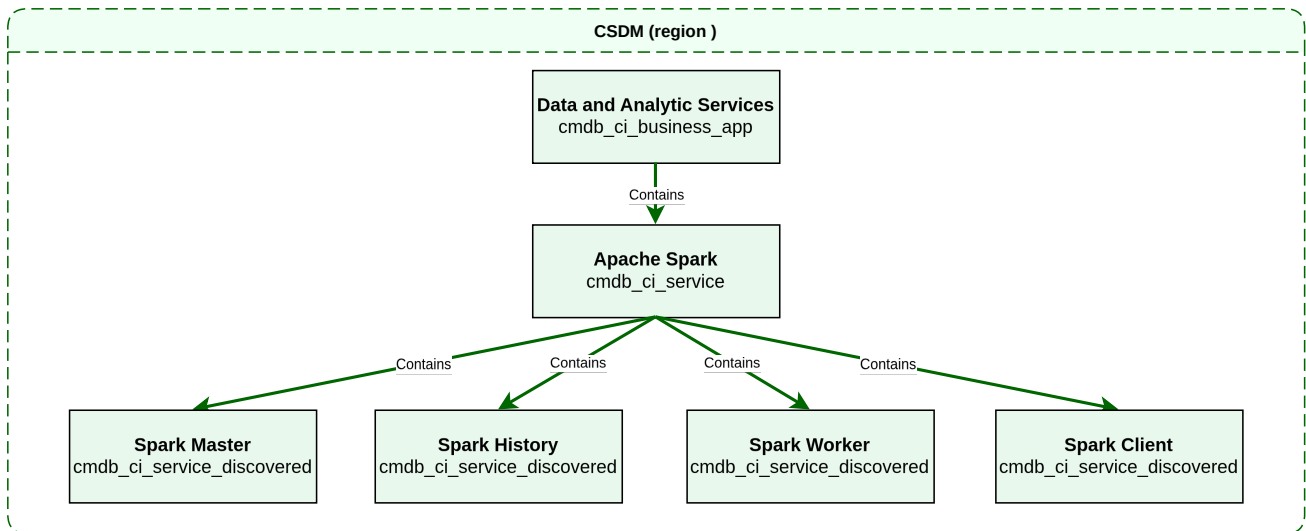


Figure 6. Spark CSDM Model

There is a service instance each of the materialized Spark services: the Spark Master, the Spark Workers, and the Spark History Server. The Spark Master is the central point of control for the Spark application. The Spark Workers are the nodes that do the actual work of the application. The Spark History Server is a built-in observability component that provides debugging and monitoring capabilities for Spark. The forth service instance is a synthetic service instance that represents all Spark clients that talk directly to a the Spark Workers without a using a Spark Master. We will discuss this further in the [Client-side Model](#) section.

You must manually create each one of the CSDM CIs in ServiceNow. To simplify the tedium of creating the CSDM CIs, we have used a tool to create the CIs based on a YAML specification. While the tool is beyond the scope of this document we share the YAML specification in [Spark CSDM Model for the Spark Master](#), below. It illustrates most of the key CI attributes needed for the Spark application.

Listing 2. Spark CSDM Model for the Spark Master

```

business_applications:
- name: Data and Analytic Services
  identifier: data-and-analytic-services
  short_description: Data processing and analytics platform services
  operational_status: "1"
  active: "true"
  business_owner: gbrooks
  it_application_owner: gbrooks

business_services:
- name: Apache Spark
  identifier: apache-spark
  short_description: Distributed data processing cluster
  operational_status: "1"
  parent_business_application: Data and Analytic Services
  owned_by: gbrooks
  business_criticality: "4 - not critical"

service_instances:
- name: Spark Master
  identifier: spark-master
  short_description: Spark master - Kubernetes StatefulSet
  operational_status: "1"
  parent_business_service: Apache Spark

```

```

platform: kubernetes
service_mapping: tags
discover: false
environment: on-prem
location: brooks-lab
owned_by: gbrooks
business_criticality: "4 - not critical"
depends_on:
  - OpenTelemetry Collector
  - Logstash
  - Elasticsearch
  - name: lab3
    type: nfs_server

```

6.2. Service-Side Model

The Service-side model used in Spark cluster mode shows the logical relationships between CSDM service instances, Kubernetes-discovered CIs, SGC-imported entities, and Event Management bindings for **service-side** logs. Log alerts bind the **service instance**, not the pod. Pods remain in the CMDB for topology; `k8s-pod-name` is a fallback lookup to the SI via `svc_ci_assoc` when `sn-service-instance` is missing.

Spark Service-Side Model illustrates the Service-side model of the Spark Master. The other Spark services, e.g. Spark Workers and the Spark History Server, would have a very similar CMDB models.

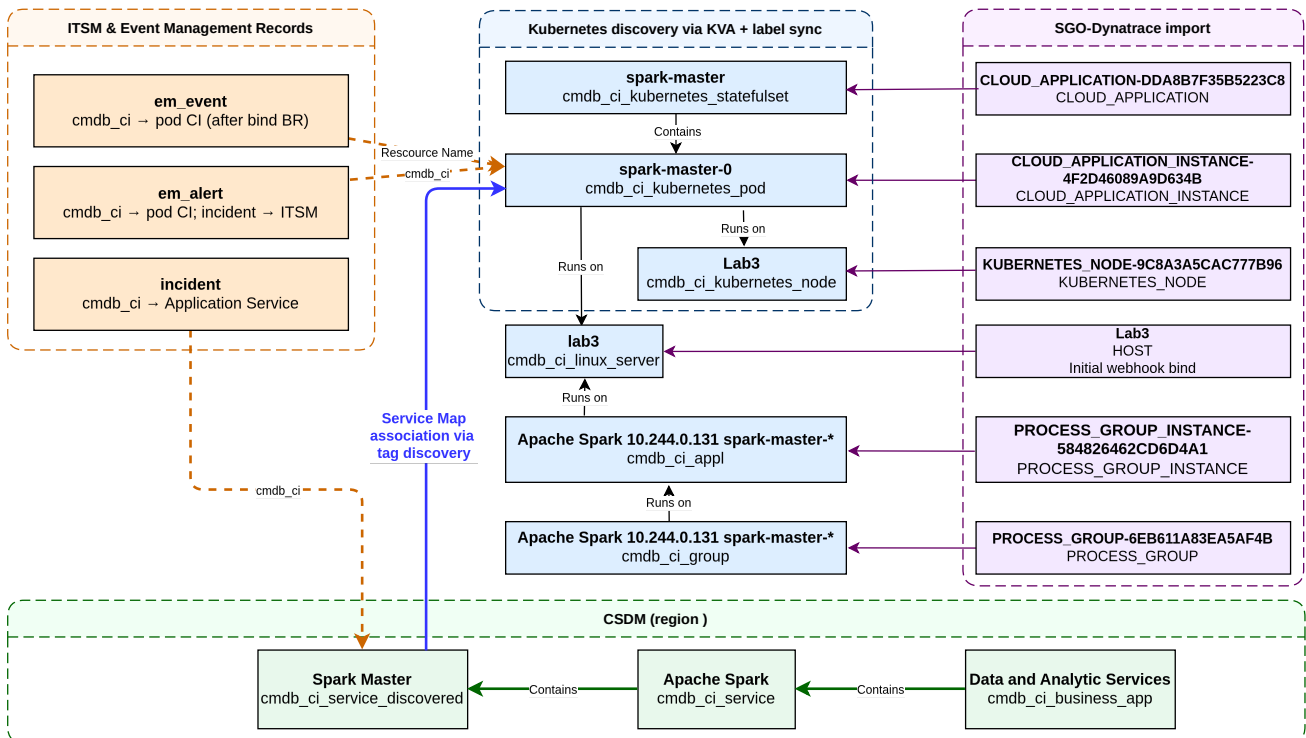


Figure 7. Spark Service-Side Model

The table below enumerates the different CMDB objects in the Service-side scenario. For those CIs that Dynatrace can generate or that you can correlate to a CI, the corresponding Dynatrace entity type is shown. The last column summarizes how Service Graph Connector (SGC / **SGO-Dynatrace**) correlates the Dynatrace entity to the CMDB CI. Key in this process is what the Identity and Reconciliation Engine (IRE) does to merge the Dynatrace entity with the CMDB CI.

Table 4. Service-side CMDB objects and SGC correlation

CI Name	CI Class	DT Entity	SGC correlation to CI
spark-master	cmdb_ci_kubernetes_statefulset	CLOUD_APPLICATION	SGC Kubernetes workload import maps CLOUD_APPLICATION → workload CI; entityId stored in sys_object_source (name=SGO-Dynatrace). IRE merges with the KVA StatefulSet when names match.
spark-master-0	cmdb_ci_kubernetes_pod	CLOUD_APPLICATION_INSTANCE	SGO-Dynatrace Kubernetes Pod feed maps CLOUD_APPLICATION_INSTANCE → cmdb_ci_kubernetes_pod ; entityId → sys_object_source.id . IRE merges with the KVA pod when name matches.
lab3	cmdb_ci_kubernetes_node	KUBERNETES_NODE	SGO-Dynatrace Kubernetes Node feed maps KUBERNETES_NODE → cmdb_ci_kubernetes_node ; entityId → sys_object_source.id . Merges with the KVA node CI on node name.
lab3	cmdb_ci_linux_server	HOST	SGO-Dynatrace Hosts feed maps HOST → computer / cmdb_ci_linux_server ; entityId → sys_object_source.id . IRE merges with Horizontal Discovery on a case insensitive match of hostname or FQDN.
Apache Spark 10.244.0.131 spark -master-* - PROCESS_GROUP -6EB611A83EA5AF4B	cmdb_ci_group	PROCESS_GROUP	SGO-Dynatrace Process Groups feed maps PROCESS_GROUP → cmdb_ci_group ; entityId → sys_object_source.id .
Apache Spark 10.244.0.131 spark -master-* spark -master-* spark -master@lab3	cmdb_ci_appl	PROCESS_GROUP_INSTANCE	SGO-Dynatrace process / PGI cascade maps PROCESS_GROUP_INSTANCE → cmdb_ci_appl ; entityId → sys_object_source.id .

To be able to correlate a critical log message to the appropriate service instance, we tag the **workload controller** and the **pod template** with **servicenow.com/service-instance** (see [Kubernetes Architecture](#)). Kubernetes does not copy controller labels onto pods, so both places are required. The tag value is the **display name** of the service instance with spaces written as hyphens (e.g. **Spark Master** → **Spark-Master**). SGC 1.15.0 has **no Kubernetes Workload** data source; Deployment/StatefulSet CIs come from horizontal discovery (KVA); SOS/**correlation_id** for **CLOUD_APPLICATION** is usually empty — name lookup is required.

Listing 3. Spark Master StatefulSet — controller and pod-template labels

```
metadata:
  name: spark-master
  namespace: spark
  labels:
    app.kubernetes.io/name: spark-master
```

```

servicenow.com/service-instance: Spark-Master
spec:
  template:
    metadata:
      labels:
        app.kubernetes.io/name: spark-master
        servicenow.com/service-instance: Spark-Master

```

Tag discovery writes `servicenow.com/service-instance` to `cmdb_key_value` on the **controller** CI (from `metadata.labels`) and on each **pod** CI (from the template). Tag-based Service Mapping matches the tag value against the population rule and records membership in `svc_ci_assoc`. Log alerts use the OpenPipeline `sn-service-instance` stamp as the primary bind; `svc_ci_assoc` is the fallback.

6.3. Client-side Model

The client-side scenario is of interest on its own as there are so many workstation and mobile client applications. In the client-side scenario the log message occurs on workstation or mobile device that is not managed in the CMdb. As ServiceNow does not manage the client device in the CMdb, there are no CIs to attach to any log alert or incident. One way to address this is to create a synthetic CI that represents the application running on any client device. For this we choose the service instance class(`cmdb_ci_service_discovered`) to instantiate a CI for the Spark Client. This CI will not have any infrastructure CIs associated with it as the client device is not managed in the CMdb.

Spark Client-Side Model illustrates the Client-side model of the Spark Client. It shows a device (and host) independent model consisting of a single service instance CI that represents the Spark Client.

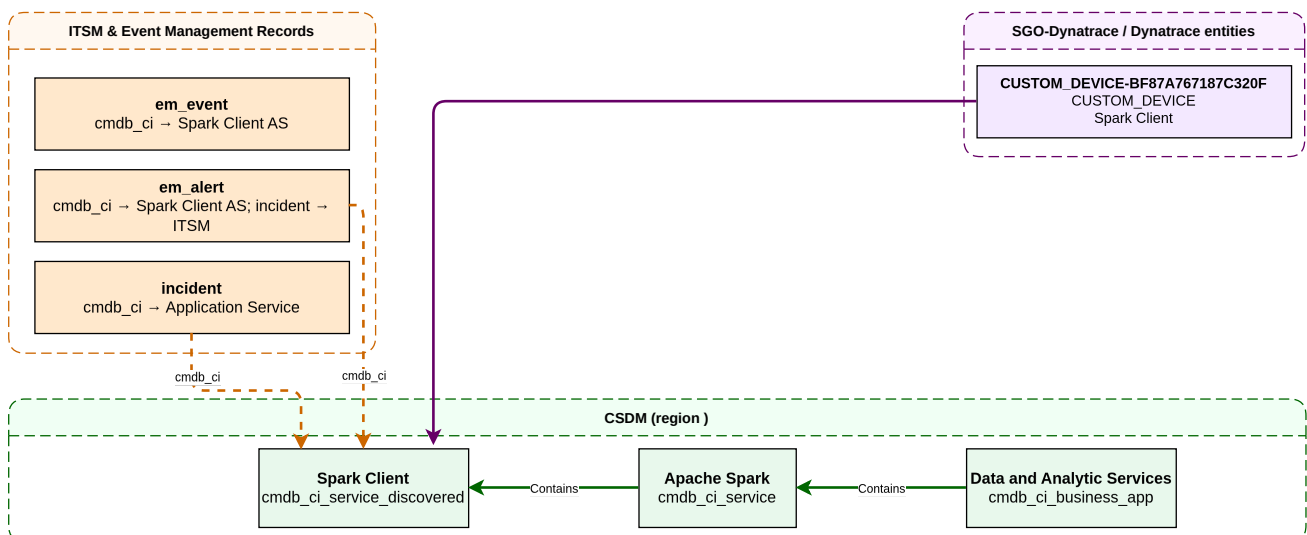


Figure 8. Spark Client-Side Model

The service instance YAML specification for the Spark Client is shown below.

Listing 4. Spark CSDM Model for the Spark Client

```

- name: Spark Client
  identifier: spark-client

```

```
platform: host
service_mapping: manual
discover: false
depends_on:
  - Spark Master
```

Now, given the Spark Client service instance, how do we map a log message on the client device to that CI? There is no Kubernetes label. The OneAgent custom log source tails `/mnt/spark/client-logs/<SPARK_DRIVER_INSTANCE>/` and stamps `service_instance`; OpenPipeline maps that to `sn-service-instance = Spark-Client`. ServiceNow enrich binds the log alert to the Spark Client service instance and overwrites the HOST SOS. A Dynatrace CUSTOM_DEVICE named “Spark Client” is **historical** — it is not the current bind.

6.3.1. Kubernetes workload labels

Pods remain in the CMDB, but **log alerts no longer use the pod as `cmdb_ci`**. Labels belong on the **controller** (`metadata.labels` → durable Deployment/StatefulSet CI) **and** on the pod template (copied onto each pod). See [Why the same labels appear on the controller and on the template](#).

Tag-based Service Mapping matches workload CIs to a service instance using `cmdb_key_value` rows copied from those labels. The ServiceNow label uses the vendor-domain prefix `servicenow.com/`. This implementation uses the single key listed below.

Table 5. ServiceNow label keys for tag-based Service Mapping

Key	Role
<code>servicenow.com/service-instance</code>	The only tag filter. Value equals the service instance display name (for example <code>Spark-Master</code> in the Kubernetes label character set). Tag-based Service Mapping records matching controller CIs (and pods, from the template) as members of that service instance in <code>svc_ci_assoc</code> .

The `servicenow.com/service-instance` tag is required on every workload that belongs to a tag-mapped service instance; no other ServiceNow tags (environment, location) are used. The tag value must match the display name of the service instance CI (spaces written as hyphens for Kubernetes labels). This is a naming contract that the teams need to keep in mind: renaming a service instance requires updating the workload labels and the tag population rule together.

[Spark Worker Deployment — controller and pod-template labels](#) illustrates the Spark Worker Deployment controller and pod-template labels.

Listing 5. Spark Worker Deployment — controller and pod-template labels

```
metadata:
  name: spark-worker-lab1
  namespace: spark
  labels:
    app.kubernetes.io/name: spark-worker
    servicenow.com/service-instance: Spark-Worker
spec:
  template:
    metadata:
      labels:
        app.kubernetes.io/name: spark-worker
```

6.3.2. Tag-Based Service Mapping and svc_ci_assoc

Getting a pod (or container) associated with its service instance is a meeting of two halves that are configured independently and joined by ServiceNow:

1. **The workload side.** The Deployment/StatefulSet manifests carry `servicenow.com/service-instance` on the **controller** and on the **pod template**. KVA copies those labels into `cmdb_key_value` on the controller CI and on each pod CI.
2. **The service side.** When the CSDM specification is deployed (`csdm/deploy.yml`), every service instance with `service_mapping: tags` gets a **tag population rule**: the deploy calls the Service Mapping Operations REST API (`/populate_tags`), which invokes `SMServByTagsUtils.updateServiceFromTagsList()` against the service instance `sys_id` with the single tag predicate `servicenow.com/service-instance = <display name>`.

ServiceNow's tag-based Service Mapping engine then evaluates the population rule against `cmdb_key_value` and records each matching workload CI as a member of the service instance in the `svc_ci_assoc` table (Service CI Association). This membership — not a CMDB relationship — is what incident automation traverses.

Two design rules follow from this model:

No CMdb relationship between service instances and workloads: Runtime topology relationships (pod → node → host, workload → pod) are imported by the Service Graph Connector from Dynatrace's Smartscape view and are realized in ServiceNow as CMDB relationships. However, Service-instance ownership lives only in a service map (i.e. on the `svc_ci_assoc` table), maintained by tag-based Service Mapping. * **Elasticity with CSDM modeling:** Once the service instance and its population rule are created during CSDM modeling, new replicas of a known workload (for example a scaled-out Spark Worker Deployment) are picked up by KVA and joined to the service instance by Service Mapping alone. CSDM Modeling is needed only when an **entirely new** service instance is introduced — a controlled CSDM onboarding step and not at runtime.

6.3.3. ServiceNow — SGC Topology Import

Table 6. SGC topology import — Dynatrace entity to CMDB class

SGC scheduled import	Dynatrace entity type	CMDB class	Role in log incidents
Hosts	HOST	cmdb_ci_linux_server	Infra alert CI for host CPU (SOS). Not the log alert CI.
Kubernetes Pod	CLOUD_APPLICATION_INSTANCE	cmdb_ci_kubernetes_pod	Topology / fallback <code>k8s-pod-name</code> lookup — not the log alert CI
Kubernetes workload (KVA; not SGC 1.15.0)	CLOUD_APPLICATION	cmdb_ci_kubernetes_deployment / statefulset	Infra alert CI for OOTB workload problems; SOS usually empty — bind by name

SGC scheduled import	Dynatrace entity type	CMDB class	Role in log incidents
Process Groups	PROCESS_GROUP	cmdb_ci_appl	Alternative when Davis binds to JVM process
Application Relationships	Smartscape edges	cmdb_rel_ci	Optional topology context

Each imported entity receives a **sys_object_source** row: name=SGO-Dynatrace, id=<Dynatrace entityId>, target_sys_id → CMDB CI sys_id.

Configure on the ServiceNow instance:

- Service Graph Connector for Dynatrace Observability (i.e.the scoped app 'sn_dynatrace_integ')
- Inbound event listener: POST /api/sn_em_connector/em/inbound_event?source=SGO-Dynatrace
- Scheduled SGC discovery imports for hosts and process groups. Kubernetes **workload** CIs come from KVA / horizontal discovery (SGC 1.15.0 has no Workload data source).

6.3.4. Dynatrace — management zone

Place hosts, Kubernetes cluster entities, and pods that run the application in a **management zone** dedicated to the integration (for example **Application Observability**). The **alerting profile** for ServiceNow problem notifications must scope to that management zone so only in-scope problems are forwarded.

Chapter 7. Spark Application - Stage 2

The development and operation teams need to configure how the Spark application is deployed and how it is logging. This is done in two areas. One area is to specify the association between the Spark execution context and the CSDM service instance for the Spark application. The other area is to configure the Spark application logging. Both configurations depend on which Spark mode is being used: Cluster Mode or Client Mode.

7.1. Service Instance Association

Service instance association is done in two significantly different ways depending on the Spark mode. For cluster mode we can use Kubernetes labels. For client mode we need to use a custom log source path in OneAgent to identify the service instance.

7.1.1. Cluster Mode

In cluster mode, associate each Spark workload with its CSDM service instance by setting `servicenow.com/service-instance` on the **controller** (`metadata.labels`) **and** on the **pod template**. Kubernetes does not copy controller labels onto pods. The label value is the service instance **display name** in the Kubernetes label character set (for example `Spark Master` → `Spark-Master`). OpenPipeline maps the running pod-name prefix (`spark-master-`, `spark-worker-`, `spark-history-*`) to `sn-service-instance`.

Listing 6. Spark Master StatefulSet — controller and pod-template labels

```
metadata:
  name: spark-master
  namespace: spark
  labels:
    app.kubernetes.io/name: spark-master
    servicenow.com/service-instance: Spark-Master
spec:
  template:
    metadata:
      labels:
        app.kubernetes.io/name: spark-master
        servicenow.com/service-instance: Spark-Master
```

7.1.2. Client Mode

In client mode the Spark Driver runs on a workstation (or a host used as a workstation). There is no Kubernetes label. The custom log source tails `/mnt/spark/client-logs/<SPARK_DRIVER_INSTANCE>/` and stamps `service_instance`; OpenPipeline maps that to `sn-service-instance = Spark-Client`.

7.2. Log File Generation

In either mode, the Spark application will generate log files using Log4j2. However, where those log files are written is significantly different. For cluster mode, the log files are written to the

container stdout. This is the container best practice. For client mode, where the Spark driver is generating the logs, the logs are written to a file under `/mnt/spark/client-logs/`.

To start with, [Spark Log File Sample](#) illustrates a sample Log4j line (same pattern for Client and Cluster Mode).

Listing 7. Spark Log File Sample

```
2026-07-20 12:40:19 WARN  Utils:250 - Your hostname, Lab3, resolves to a loopback address: 127.0.1.1; using
192.168.1.207 instead (on interface enp3s0)
2026-07-20 12:40:19 WARN  Utils:244 - Set SPARK_LOCAL_IP if you need to bind to another address
2026-07-20 12:40:27 WARN  TaskSetManager:250 - Lost task 0.0 in stage 0.0 (TID 0) (10.244.2.145 executor 0):
org.apache.spark.SparkException: [FAILED_READ_FILE.FILE_NOT_EXISTS
T] Encountered error while reading file file:///mnt/spark/data/elements/Periodic_Table_Of_Elements.csv. File does not
exist. It is possible the underlying files have been up
dated.
2026-07-20 12:40:29 ERROR TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job
```

As typical with many applications, there is not a lot of structure to the log files. There is the ever present time stamp, an error level, the file and line number of the log message, and the message itself. Of particular importance is the log level. Instead of matching a long list of critical log message patterns, we can use it to decide if the log message should be considered a critical issue. For the reference implementation we choose to use both the WARN and ERROR levels. We use warnings for convenience as they are easier to generate.

There are several key questions we need to answer:

- Where will the log lines be emitted (file path vs container stdout)?
- What key metadata will we need to extract from the log lines?
- What other key metadata will we need about the log context (mode, instance, pod)?

7.2.1. Cluster Mode

Cluster Mode Spark components (Master, Workers, History Server, executors) are Kubernetes pods that emit application logs to **stdout**. This is the best practice, because the OneAgent Log module can ingest the log entries with the Kubernetes entity context.

7.2.1.1. Log emission

When containers managed by Kubelet write to **stdout** and **stderr**, the container runtime writes these streams to structured log files on the host node filesystem. Kubelet organizes these logs under `/var/log/pods/<namespace>_<pod_name>_<pod_uid>/<container_name>/` on the host node. From their, OneAgent can tail the logs and ingest them with the Kubernetes entity context.

7.2.1.2. Log4j configuration

Cluster Mode looks for the Log4j configuration file in `spark/conf/log4j2-cluster.properties`. This file needs to be configured as a Console appender.

Listing 8. Log4j Console — Cluster Mode

```
# ConsoleAppender - kubelet /var/log/pods → Dynatrace Container Output
appender.console.type = Console
appender.console.name = Console
appender.console.layout.type = PatternLayout
appender.console.layout.pattern = %d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n%ex

# Root logger - Console only (container stdout)
rootLogger.level = info
rootLogger.appenderRefs = console
rootLogger.appenderRef.console.ref = Console
```

7.2.2. Client Mode

Client mode works is significantly different. The Spark driver is running on a client device as a standalone application. There can be multiple drivers running on the same host. Unlike with Kubernetes, there is no well-defined enforcement of where the log files are written. The caller of the Spark driver is responsible for managing the location of the log files.

7.2.2.1. Log emission

Typically, you will use a script to marshall all the configurations for the Spark driver and then run the driver (PySpark). By convention, we expect the log files to be written to a node-local directory under `/mnt/spark/client-logs/`. By default, the logs will be written to a directory named after the PID of the script. Since, there can be multiple drivers running on the same host, the script needs to ensure unique log directories. The script must tell the Spark Driver where to find the Log4j configuration file and it must ensure the log directory is set. [Spark client-mode log directories](#) illustrates the client-mode log directories.

Listing 9. Spark client-mode log directories

```
/mnt/spark/client-logs/      # Client mode (node-local)
├── 12345/                  # default: wrapper PID
├── par-a/                  # SPARK_DRIVER_INSTANCE override for parallel runs
└── par-b/
```

7.2.2.2. Log4j configuration

OneAgent harvests the RollingFile tree as shown in [Log4j RollingFile — client-side](#), below.

Listing 10. Log4j RollingFile — client-side

```
appender.rolling.type = RollingFile
appender.rolling.name = RollingFile
appender.rolling.fileName = ${env:SPARK_LOG_DIR:-/mnt/spark/client-logs/default}/spark-app.log
appender.rolling.filePattern = ${env:SPARK_LOG_DIR:-/mnt/spark/client-logs/default}/spark-app-%d{yyyy-MM-dd}-%i.log
appender.rolling.filePermissions = rw-r--r--
appender.rolling.layout.type = PatternLayout
appender.rolling.layout.pattern = %d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n%ex
appender.rolling.policies.type = Policies
appender.rolling.policies.time.type = TimeBasedTriggeringPolicy
appender.rolling.policies.time.interval = 1
appender.rolling.policies.size.type = SizeBasedTriggeringPolicy
appender.rolling.policies.size.size = 20MB
appender.rolling.strategy.type = DefaultRolloverStrategy
```

```
appender.rolling.strategy.max = 10
```

OneAgent will then need to be configured to harvest the log files under the directory `/mnt/spark/client-logs/*`, where the subdirectory is the log directory for the Spark driver.

Chapter 8. Dynatrace

This section describes the steps to send logs from an application to Dynatrace and then the steps to send the problems to ServiceNow and store the events in the Grail bucket. Before we begin this detailed process, we need to begin with the payload that will be sent to ServiceNow.

8.1. Problem Payload

The problem notification payload that is sent to ServiceNow contingent on an event matching a Davis watcher clause in one of the Davis processors. Once an event matches a watcher clause, Davis will create a problem. Depending on the alerting profile, Davis may associate multiple events to the problem. Eventually, Davis will send the problem to ServiceNow to create an event in ServiceNow. The payload is sent to ServiceNow via the SGO webhook. This is a different webhook than older, legacy webhook supported by ServiceNow. [Problem notification payload \(SGC\)](#) shows the top-level schema of the problem notification payload.

Listing 11. Problem notification payload (SGC)

```
{
  "ConnectionId": "<connections_alias_sys_id>",
  "ImpactedEntities": {ImpactedEntities},
  "ImpactedEntity": "{ImpactedEntity}",
  "PID": "{PID}",
  "ProblemDetailsJSON": {ProblemDetailsJSON},
  "ProblemDetailsText": "{ProblemDetailsText}",
  "ProblemID": "{ProblemID}",
  "ProblemImpact": "{ProblemImpact}",
  "ProblemSeverity": "{ProblemSeverity}",
  "ProblemTitle": "{ProblemTitle}",
  "ProblemURL": "{ProblemURL}",
  "State": "{State}",
  "Tags": "{Tags}"
}
```

The placeholder values of the form `{<placeholder>}` are replaced with runtime values determined by Dynatrace. If a placeholder value is in double quotes, the value will be a string. Those placeholder values without double quotes will be a nested JSON structure.

Dynatrace and application developers can influence how Dynatrace derives some of the values in a problem field. [Table 7, “Problem notification payload template fields”](#) describes how Dynatrace fills in these placeholder values. The table also indicates which fields a developer may influence.

Table 7. Problem notification payload template fields

Field Name	Source	Value of the field
ProblemID	Dynatrace	The human-readable display id of the problem, like P-260729. Dynatrace assigns this value.
PID	Dynatrace	Dynatrace assigns this field to an internal unique problem system for use in APIs and URLs.

Field Name	Source	Value of the field
ProblemTitle	Influenceable	Dynatrace derives this short problem title primarily from the Davis event's event.name attribute. When multiple events are associated with (merged into) one problem, Davis builds it from those event names.
ProblemSeverity	Influenceable	Dynatrace computes the problem severity class, e.g., CUSTOM_ALERT , from event.type on the events in the problem. For multi-event problems, Dynatrace aggregates to the problem's severity class. See Dynatrace Event Types and Severity Levels (pdt20158) for the full list of allowed event.type values.
ProblemImpact	Influenceable	Dynatrace derives the Problem's impact tier, for example APPLICATION , from the Smartscape layer of the impacted entities. These entities come from the Davis event's dt.source_entity and the related topology. For a full list of the tiers and related entities see Problem Impact Tiers .
ProblemURL	Dynatrace	Dynatrace sets this field to a deep link to the problem in the Dynatrace UI. Dynatrace builds this from the tenant base URL and the problem id.
State	Dynatrace	Dynatrace sets this from the problem lifecycle (OPEN or RESOLVED) at send time (problem still open vs closed).
Tags	Influenceable	Dynatrace sets the tags to a comma-separated tags of the form <tag>:<value> from tags stamped on the impacted entities. Software and Dynatrace developers can stamp tags on an entity through OneAgent, the Dynatrace UI, the Dynatrace API, and container deployment descriptors. Dynatrace derives the list by collecting tags defined on the problem's impacted monitored entities. It is not a top-level OpenPipeline field on the Davis event.
ImpactedEntity	Influenceable	Dynatrace sets the impacted entity to a summary string of impacted entity's name. Dynatrace derives it by formatting the display names of the same entities that populate ImpactedEntities (typically a comma-separated name list).
ImpactedEntities	Influenceable	Dynatrace sets impacted entities to a JSON array of the impacted entities. Each element of the array is an object with the fields type , name , and entity . Dynatrace derives the array from the problem's impacted-entity set: the Davis event's source and related entities (dt.source_entity) plus Smartscape relationships Davis uses when expanding impact.
ProblemDetailsText	Influenceable	Human-readable problem details including root-cause narrative. Dynatrace generates this text from the problem's event details—chiefly event.description and related event text such as event.name . Custom properties appear when included in those templates.
ProblemDetailsJSON	Influenceable	Dynatrace sets this to the JSON object representing the full problem as per the Problems API v1 schema. This object includes custom properties that an OpenPipeline can stamp on the Davis events in the problem. This provides a key extension mechanism for OpenPipeline developers to add custom properties to the problem. See Problem Details JSON for details.

8.1.1. Problem Details JSON

The **ProblemDetailsJSON** field is a nested JSON structure that contains a lot of information about the problem. It is the same JSON structure that is used to create a problem within Dynatrace.

ProblemDetailsJSON contains summary information, details about the impacted entities, and about entity tags, the events grouped into the problem, and custom properties of the events. The custom properties provide a key mechanism to pass additional information to ServiceNow.

The top-level shape of **ProblemDetailsJSON** (Problems API v1) is shown in [ProblemDetailsJSON top-level structure](#) (SGC / Problems API v1), below. Nested arrays and objects such as **tagsOfAffectedEntities** are summarized here and described in later subsections.

8.1.2. Problem Severity

Severity is a categorization of events and problems into seven categories. The categories are based on qualitative characteristic of the monitor generating the event or problem. For example, a performance oriented monitor would have a severity of "PERFORMANCE". Some performance monitors could have higher or lower importance (aka severity) than a different class of severity, like ERROR. The importance of a monitor really depends on what the monitor is monitoring. Even so, you can order the severity categories by decreasing level of severity. [Dynatrace Problem Severity Levels](#) shows the seven severity categories ordered by decreasing level of severity.

Table 8. Dynatrace Problem Severity Levels

Severity Level	Severity Level Code	Description
1	MONITORING_UNAVAILABLE	Widespread monitoring interruption: the majority of installed OneAgents lose their connection with the Dynatrace server. This usually shows up as a lack of visibility for both availability and performance monitoring.
2	AVAILABILITY	High-severity incidents within the environment, such as a complete outage or unavailability of servers or processes.
3	ERROR	Increased error rates or other error-related incidents that interfere with the regular operation of the environment. Alerting profiles match this class as ERRORS (plural); problem and webhook fields usually use ERROR (singular).
4	PERFORMANCE	Slowdown: a decrease in performance of an operational service or application. Less severe than error or availability events, but still indicates potential service-performance issues.
5	RESOURCE_CONTENTION	Resource contention. Typical examples include CPU saturation and memory saturation.
6	CUSTOM_ALERT	Custom alerts for user-defined thresholds (any Dynatrace metric) or OpenPipeline Davis events of this class. Like other problem-opening severities, they can be controlled by Dynatrace Intelligence and alerted on.
7	INFO	Informational or warning-class events that do not result in creation of a new problem on their own—for example important deployments, configuration changes, administrative events (such as automatic VM migration), and WARNING event types. These do not indicate an abnormal situation by themselves and are not sent as problem alerts.

A problem's severity is determined by the severity of the Davis events that are grouped into the

problem. The severity of a Davis event is determined by the `event.type` property of the event. Event types are mapped into severity levels. For problem severities, [Dynatrace Event Types and Severities](#) shows the mapping of event types to problem severities.

Dynatrace developers can influence the severity of the events within a problem, which in turn influence the severity of the problem. Developers do this by setting the custom property `event.type` on each Davis event to one of the allowed event types. There is a wide range of event types. [Event Types](#) lists every event type on the tenant. Many of these are set in out-of-the-box alerting profiles. Within a Davis processor, only a subset of event types are allowed. [Dynatrace Davis Event Types \(OpenPipeline / custom ingest\)](#) summarizes those event types that a developer can set in an OpenPipeline.

Table 9. Dynatrace Davis Event Types (OpenPipeline / custom ingest)

Event Type Name	Allowed Event Types	Create Problem	Description and Usage
Availability event	<code>AVAILABILITY_EVENT</code>	Yes	High-severity outage / unavailability class. Opens a problem with severity bucket Availability (alerting profile <code>AVAILABILITY</code> , webhook <code>ProblemSeverity</code> typically <code>AVAILABILITY</code>). Use for custom “entity/service down” signals.
Error event	<code>ERROR_EVENT</code>	Yes	Generic error class (custom error event). Opens a problem with severity bucket Error (profile <code>ERRORS</code> , webhook typically <code>ERROR</code>). Prefer for application/runtime failures. Merge allowed by default; set <code>dt.davis.is_merging_allowed=false</code> when you need isolated problems.
Performance event	<code>PERFORMANCE_EVENT</code>	Yes	Slowdown / degraded-performance class. Opens a problem with severity bucket Performance / Slowdown (profile <code>PERFORMANCE</code> , webhook typically <code>PERFORMANCE</code>). Use for latency, response-time, or throughput degradation signals.
Resource contention event	<code>RESOURCE_CONTENTION_EVENT</code>	Yes	Resource-pressure class (CPU, memory, disk, network contention). Opens a problem with severity bucket Resource (profile <code>RESOURCE_CONTENTION</code> , webhook typically <code>RESOURCE_CONTENTION</code>). Use for custom resource saturation events (OOTB host/K8s anomalies often land here too).
Custom alert	<code>CUSTOM_ALERT</code>	Yes	User-defined threshold / metric-style alert class. Opens a problem with severity bucket Custom (profile <code>CUSTOM_ALERT</code> , webhook typically <code>CUSTOM_ALERT</code>). Prefer for “queue depth / custom metric breached” style alerts and for OpenPipeline log-extracted Davis events (Dynatrace tutorial default). Merge disallowed by default.
Warning	<code>WARNING</code>	No	Warning / near-term risk. Use to annotate risk without notifying via problem profiles.
Custom info	<code>CUSTOM_INFO</code>	No	Informational marker (e.g. load test start). Use for operational context correlated into later problems.
Custom annotation	<code>CUSTOM_ANNOTATION</code>	No	Highlights an interesting time window for triage. Use for third-party annotations on entities/timelines.

Event Type Name	Allowed Event Types	Create Problem	Description and Usage
Custom configuration	CUSTOM_CONFIGURATION	No	Reports a configuration change. Use for change-correlation context.
Custom deployment	CUSTOM_DEPLOYMENT	No	Reports a software deployment (version, owner, etc.). Use for CI/CD → Davis change correlation.
Marked for termination	MARKED_FOR_TERMINATION	No	Planned shutdown / decommission of an entity. suppresses availability alerting for a period (about 60 minutes) so intentional takedowns do not raise false outages. Use before graceful host/process removal.

The `event.type` property must be set to one of the codes in the above table. As you can see from the table, event types are partially an event category (type) and partially a hierarchy of severity levels. It is not suited for indicating to ServiceNow a graduated (high, medium, low) level of severity. *We will alternative means to communicate severity to ServiceNow in a problem.*

However, we still must set `event.type` within Dynatrace. Dynatrace recommends setting `event.type` for logs to "CUSTOM_ALERT" or to "ERROR_EVENT". Even within Dynatrace there is not an established best practice. For using "CUSTOM_ALERT" see [Extracting Davis Events from Logs](#). For using `ERROR_EVENT` see [Detecting Problems with Logs](#) or [Problem Detection with Logs](#). In our case we will use "CUSTOM_ALERT".

To further complicate matters, while problem severities are derived from `event.type`, they are not always spelled the same. [Dynatrace Event Types and Severities](#) shows the mapping of the allowed event types to problem severities, for those event types that create a problem.

Table 10. Dynatrace Event Types and Severities

Event Type Name	Event Type <code>event.type</code>	Alerting Profile <code>severityLevel</code>	Webhook Payload <code>ProblemSeverity</code>
Availability	AVAILABILITY_EVENT	AVAILABILITY	AVAILABILITY
Error	ERROR_EVENT	ERRORS	ERROR
Slowdown	PERFORMANCE_EVENT	PERFORMANCE	PERFORMANCE
Resource	RESOURCE_CONTENTION_EVENT	RESOURCE_CONTENTION	RESOURCE_CONTENTION
Custom	CUSTOM_ALERT	CUSTOM_ALERT	CUSTOM_ALERT

See [Event categories](#) and [alerting.profile schema](#). for more details.

8.1.3. Impact Tier

The impact tier is the Smartscape layer of the entity that is impacted by the problem.

Table 11. Problem Impact Tiers

Smartscape Tier	Description
APPLICATION	The Smartscape layer entities that denote the objects that implement the User-facing components. Representative entity types include APPLICATION (RUM web application), MOBILE_APPLICATION , and CUSTOM_APPLICATION . Synthetic front ends (SYNTHETIC_TEST , HTTP_CHECK) may also surface here when they are the impacted entities.
SERVICE	The Smartscape layer entities that denote the objects that implement services. Representative Dynatrace entities include SERVICE (monitored service / call endpoint in Smartscape).
INFRASTRUCTURE	The Smartscape layer entities that denote the objects that implement applications and services. Representative Dynatrace entity types include HOST , HOST_GROUP , PROCESS_GROUP , PROCESS_GROUP_INSTANCE , DISK , NETWORK_INTERFACE , CONTAINER_GROUP , CONTAINER_GROUP_INSTANCE , CUSTOM_DEVICE , KUBERNETES_CLUSTER , KUBERNETES_NODE , CLOUD_APPLICATION_NAMESPACE , CLOUD_APPLICATION (workload controller), CLOUD_APPLICATION_INSTANCE (pod), and KUBERNETES_SERVICE .

Lab Client file-tail and host CPU problems typically land here (**HOST**); Cluster Mode workload/pod problems typically land here (**CLOUD_APPLICATION** / **CLOUD_APPLICATION_INSTANCE**).

8.1.4. Problem Details

The **ProblemDetailsJSON** field is a nested JSON structure that contains the full representation of the problem. It uses the Dynatrace V2 problem API schema to represent the problem. The **ProblemDetailsJSON** field contains a lot of information about the problem including summary information, details about the impacted entities, and about entity tags, the events grouped into the problem, and custom properties of the events. *The custom properties provide a key mechanism to pass additional information to ServiceNow.* The [ProblemDetailsJSON top-level structure \(SGC / Problems API v1\)](#) shows the top-level structure of **ProblemDetailsJSON**.

Listing 12. ProblemDetailsJSON top-level structure (SGC / Problems API v1)

```
{
  "id": "{id}",
  "startTime": {startTime},
  "endTime": {endTime},
  "displayName": "{displayName}",
  "impactLevel": "{impactLevel}",
  "status": "{status}",
  "severityLevel": "{severityLevel}",
  "commentCount": {commentCount},
  "tagsOfAffectedEntities": [ {tagsOfAffectedEntities} ],
  "rankedEvents": [ {rankedEvents} ],
  "rankedImpacts": [ {rankedImpacts} ],
  "affectedCounts": {affectedCounts},
  "recoveredCounts": {recoveredCounts},
  "hasRootCause": {hasRootCause}
}
```

[ProblemDetailsJSON top-level fields](#) describes how Dynatrace populates these top-level fields. Complex nested structures and arrays ([tagsOfAffectedEntities](#), [rankedEvents](#), [rankedImpacts](#), [affectedCounts](#), [recoveredCounts](#)) are outlined only at this level and detailed in subsequent subsections.

Table 12. ProblemDetailsJSON top-level fields

Field Name	Source	Value of the field
id	Dynatrace	Dynatrace assigns an internal unique problem system id in the same family as the webhook payload's PID .
startTime	Dynatrace	Dynatrace sets the problem's start time to the start time of the earliest contributing event in rankedEvents . The start time is expressed as a Unix epoch millisecond timestamp.
endTime	Dynatrace	Dynatrace sets the problem's end time to the end time of the latest contributing event in rankedEvents . The end time is expressed as a Unix epoch millisecond timestamp. If the problem is still open, the end time is set to -1 .
displayName	Dynatrace	Dynatrace sets the display name to a human-readable problem id display name in the same family as the parent ProblemID , e.g. P-26082165 .
impactLevel	Influenceable	Dynatrace sets the impact tier of the problem. It uses the same derivation as the webhook payload's ProblemImpact . See Impact Tier for the different impact tiers.
status	Dynatrace	Dynatrace sets the problem's lifecycle status to OPEN or CLOSED at the time the problem is serialized. It is similar to the webhook's payload field State .
severityLevel	Influenceable	Problem severity class (e.g. ERROR , CUSTOM_ALERT). Same derivation as webhook ProblemSeverity from event.type on events in the problem. See Dynatrace Event Types and Severity Levels (pdt20158) .
commentCount	Dynatrace	Dynatrace counts the number of comments on the problem in Dynatrace and sets this field to the count.
tagsOfAffectedEntities	Influenceable	Dynatrace sets the tags to a JSON array of the distinct union of entity tags on impacted entities. The nested tag objects are described in Tags Of Affected Entities .
rankedEvents	Influenceable	Dynatrace sets the ranked events to a JSON array of all the Davis events merged into the problem. Dynatrace builds this from the problem's ranked event list. <i>Developers can influence the nested event custom properties field by setting the custom properties in an OpenPipeline.</i> See Ranked Events for details.
rankedImpacts	Dynatrace	Dynatrace sets ranked impacts to a JSON array summarizing the severity type and impact tier of the entities of the impacted entities in the problem. See Ranked Impacts for details.
affectedCounts	Dynatrace	Dynatrace counts the total number of entities involved in the problem per impact tier and sets the affected counts to a structure of the counts per impact tier. Dynatrace computes these counts from the problem's impact analysis. See Problem Impact Tiers for the impact tiers.

Field Name	Source	Value of the field
<code>recoveredCounts</code>	Dynatrace	Dynatrace counts the the number of entries with open events in the problem per impact tier. Dynatrace then sets this field to a structure with an open count for each impact tier. Object counting recovered entities by impact tier (same keys as <code>affectedCounts</code>). Dynatrace updates these as entities recover / the problem closes. Object shape is described subsequently.
<code>hasRootCause</code>	Dynatrace	Dynatrace sets this Boolean indicating depending on whether Davis identified a root-cause entity for the problem. Dynatrace sets this from problem analysis; not stamped from OpenPipeline.

8.1.5. Ranked Events

The `rankedEvents` field is a JSON array of the Davis events that Dynatrace Davis' engine grouped into a problem. The Davis engine processes candidate events and groups them into problems if several criteria are met.



Need to define the criteria either here or in a separate section. Also need to define the merge criteria and the ranking criteria.

As with the enclosing ProblemDetailsJSON object, each ranked-event element uses the Problems API v1 schema. [Event Schema](#) shows the schema of one event. There can be one or more events in a problem.

Listing 13. Event Schema

```
{
  "startTime": {startTime},
  "endTime": {endTime},
  "entityId": "{entityId}",
  "entityName": "{entityName}",
  "severityLevel": "{severityLevel}",
  "impactLevel": "{impactLevel}",
  "eventType": "{eventType}",
  "status": "{status}",
  "severities": [ {severities} ],
  "isRootCause": {isRootCause},
  "annotationDescription": "{annotationDescription}",
  "correlationId": "{correlationId}",
  "customProperties": {customProperties}
}
```

[Ranked event fields](#) describes how Dynatrace populates each of these fields. The nested `customProperties` object is the Davis' processor key extension point.

Table 13. Ranked event fields

Field Name	Source	Value of the field
<code>startTime</code>	Dynatrace	Dynatrace sets the event start time to the timestamp of the first matching Davis event report. The start time is expressed as a Unix epoch millisecond timestamp.

Field Name	Source	Value of the field
<code>endTime</code>	Dynatrace	Dynatrace sets the event end time to the timestamp when the event closed (last refresh plus <code>dt.davis.event_timeout</code>). The end time is expressed as a Unix epoch millisecond timestamp. If the event is still open, Dynatrace sets this field to <code>-1</code> .
<code>entityId</code>	Influenceable	Dynatrace uses the value of <code>dt.source_entity</code> to determine the entity's id. If the value of <code>dt.davis.is_entity_remapping_allowed</code> is false, then the value of <code>entityId</code> will be the same as the value of <code>dt.source_entity</code> . If the value is <code>true</code> , Davis may pick a more preferable entity to be the focus of the event.
<code>entityName</code>	Influenceable	Dynatrace will set the entity's name to whichever entity is selected for <code>entityId</code> .
<code>severityLevel</code>	Influenceable	Dynatrace sets the event severity class (for example <code>ERROR</code>) from the Davis event's <code>event.type</code> . See Problem Severity for more details.
<code>impactLevel</code>	Influenceable	Dynatrace sets the impact tier from the Smartscape layer of <code>entityId</code> . Same derivation as the webhook payload's <code>ProblemImpact</code> . See Impact Tier . Log file events will typically result in an impact level of <code>INFRASTRUCTURE</code> .
<code>eventType</code>	Influenceable	Dynatrace copies the Davis event's <code>event.type</code> (for example <code>CUSTOM_ALERT</code>). OpenPipeline stamps this on the Davis processor. See Dynatrace Event Types and Severity Levels (pdt20158) .
<code>status</code>	Dynatrace	Dynatrace sets the event lifecycle status to <code>OPEN</code> or <code>CLOSED</code> at the time the problem is serialized.
<code>severities</code>	Dynatrace	Dynatrace sets this to a JSON array of per-sample severity records for metric-style events. For OpenPipeline log events the array is typically empty.
<code>isRootCause</code>	Dynatrace	Dynatrace sets this Boolean according to whether Davis identified this event as a root-cause contributor. Log events with merge disabled are usually <code>false</code> .
<code>annotationDescription</code>	Influenceable	Dynatrace sets this to the Davis event's <code>event.description</code> . OpenPipeline composes that description from the log line plus the <code>sn-*</code> tokens. ServiceNow also uses this text when building <code>ProblemDetailsText</code> .
<code>correlationId</code>	Dynatrace	Dynatrace assigns an opaque correlation id that ties event reports for this Davis event together. Dynatrace generates this value; it is not an OpenPipeline field.
<code>customProperties</code>	Influenceable	Dynatrace sets this to a JSON structure of key/value pairs stamped on the Davis event in an OpenPipeline. This provides a key mechanism for OpenPipeline developers to add custom properties to the Dynatrace event and then send them to ServiceNow to be incorporated into a ServiceNow event. Tag Contract). ServiceNow enrich and the incident populator read <code>rankedEvents[0].customProperties</code> .

8.1.6. Custom Properties

For an OpenPipeline **Davis event** template, Dynatrace treats properties as the event's metadata. For an OpenPipeline **Davis event** template, Dynatrace treats properties as the event's metadata. Technically, the pipeline's custom properties schema only requires a non-empty properties list; in practice a small set is **required for a useful Davis event**, and another set of properties are

strongly recommended.

In addition, the custom properties are used to pass additional information to event management systems, like ServiceNow.

See [Davis semantic dictionary](#), [Set up alerts based on events extracted from logs](#), and [Events API v2 — POST an event](#).

8.1.6.1. Required Properties

Table 14. Identity and severity (treat as required)

Property	Why
<code>event.type</code>	Required for custom/OpenPipeline events. Sets the event class and therefore the event's <code>severityLevel</code> . Must be an allowed enum value (<code>CUSTOM_ALERT</code> , <code>ERROR_EVENT</code> , and so on).
<code>event.name</code>	Human-readable title. Part of Davis identity : reports with the same <code>event.name</code> , <code>event.type</code> , and <code>dt.source_entity</code> refresh one Davis event. Official log-to-Davis tutorials always set it.
<code>dt.source_entity</code>	Entity the event attaches to (baseline for problem <code>entityId</code> / RCA). Tutorials typically set <code>{dt.source_entity}</code> so the event stays on topology. If missing or invalid, Dynatrace often maps to the environment entity. Legacy field; prefer <code>dt.smartscape_source.id</code> where you use 3rd-generation Smartscape.

Comparable Events API ingest minimum: required `eventType` and `title` ($\approx$ `event.name`); associate an entity with `entitySelector` when not supplied in properties.

8.1.6.2. Strongly Recommended Properties

Table 15. Strongly recommended

Property	Why
<code>event.description</code>	Longer human-readable text on the event/problem. Official tutorials always set it. Useful for triage detail and for tokens consumed by downstream integrations.
<code>dt.davis.event_timeout</code> <code>dt.davis.timeout</code> <code>dt.event.timeout` (classic)</code>	Keep-alive window. Default is 15 minutes ; maximum is 6 hours . If the source does not refresh within the timeout, Dynatrace closes the event and related problems.

8.1.6.3. Optional Properties

Table 16. Often set (behavior control; not mandatory)

Property	Role
<code>dt.davis.is_merging_allowed</code> <code>dt.event.allow_davis_merge</code>	Allow (<code>true</code>) or forbid (<code>false</code>) merging this event into a larger problem. For <code>CUSTOM_ALERT</code> , merging is disallowed by default.
<code>dt.davis.is_entity_remapping_allowed</code> <code>dt.event.allow_entity_remapping</code>	Allow remapping the attach entity from event metadata, or keep the configured / <code>dt.source_entity</code> target.

Property	Role
<code>dt.davis.preferred_entity_type</code>	Preferred classic entity type when remapping is enabled.
<code>event.unique_identifier</code>	Extra split key so the same extraction config does not collapse distinct instances (classic log events often use a content-based placeholder). Complements <code>event.name</code> / <code>event.type</code> / <code>dt.source_entity</code> identity.
<code>dt.event.correlation_tag</code>	Fine-grained correlation together with name, entity, and provider.
<code>dt.event.suppress_problem</code>	Store and visualize the event but do not raise a problem.
<code>dt.event.is_rootcause_relevant</code>	Include or exclude the event from root-cause analysis.

8.1.6.4. Optional Properties

Table 17. Optional Properties

Property	Role
Any non- <code>dt.</code> keys (and extra <code>dt.entity.</code>)	Custom metadata. OpenPipeline can remap log fields into these. Useful for integrations (for example ServiceNow <code>sn- tags</code>). <i>*Not</i> required by Dynatrace.

Practical OpenPipeline minimum

Matches Dynatrace's log-to-Davis tutorial:

`event.type` + `event.name` + `event.description` + `dt.source_entity`

Add timeout, merge, remapping, and/or `event.unique_identifier` when you care about lifecycle and problem splitting.

8.1.7. Ranked Impacts

8.1.8. Tags Of Affected Entities



This chapter walks the Log to Incident reference path for **both** Spark modes.

Cluster Container Output uses OpenPipeline `k8s-short-log4j2-log-alerts`. Client file tails under `/mnt/spark/client-logs/<SPARK_DRIVER_INSTANCE>/` use `standalone-short-log4j2-log-alerts`. They share one alerting profile / SGO webhook and the same ServiceNow Event Management automation in [Stage 3](#).

Within each phase subsection below, Dynatrace- and mode-independent material comes first, then **Cluster Mode**, then **Client Mode**.

8.2. ServiceNow Integration

In the previous sections, we have exhaustively defined all the information about events and problems that are sent to ServiceNow. We next need to identify what key information

ServiceNow needs to create alerts and in turn incidents. Then we can determine whether the information is consistently available in the problem payload or if there is additional information that we need to send to ServiceNow.

8.2.1. Service Instance Tagging

The ideal CI to associate to a a log message alert is the service instance of the application. The question is, given the information available in the problem payload: How can we derive the service instance from the information available?

The `dt.source_entity` is the entity that is sent to ServiceNow as the impacted entity. Form the impacted entity, ServiceNow will attempt to lookup the corresponding CI in the CMDB. For application logs, the `dt.source_entity` is (by default)the HOST entity on which the OneAgent resides. For container logs, it is the PROCESS_GROUP_INSTANCE of the container. In the former case, there is now way to derive the service instance from the HOST entity. In the latter case, it is possible to derive the service instance from the PROCESS_GROUP_INSTANCE, but the approach is somewhat problematic and because of the rate of change of the PROCESS_GROUP_INSTANCE, it is not a reliable.

A more direct and reliable approach is to tag the events in the problem with the service instance of the application. This is done by setting the `sn-service-instance` custom property on the Davis event. This property is then read by the ServiceNow event management automation and used to set the CI on the incident.

The following decisions are the contract between the observability platform and the ITSM platform for this implementation. Stage 3 repeats the same rules under [Log to Incident design decisions \(ServiceNow\)](#).

Table 18. Log to Incident design decisions

Decision	Current rule	Why
Dash keys, not dots	<code>sn-log-signature</code> , <code>sn-service-instance</code> , <code>sn-impact</code> , <code>k8s-workload-name</code>	TBAC / Glide conditions walk <code>additional_info</code> on .. A key named <code>k8s.workload.name</code> is treated as a JSON path, not one property.
Dynatrace owns initial impact/urgency	OpenPipeline and metric-event metadata stamp <code>sn-impact</code> / <code>sn-urgency</code> . OOTB Davis/K8s cannot; SN enrich maps <code>ProblemSeverity</code> .	Alert <code>severity</code> (EM 1–5) and incident <code>priority</code> (Data Lookup) are different numbers.
Log alert CI = service instance	<code>CRITICAL_LOG_EVENT</code> / <code>sn-service-instance</code> → <code>cmdb_ci</code> is the CSDM service instance. Enrich overwrites SGO HOST/pod SOS.	Operators assign from the SI, not Lab3. Incident CI is the same SI.
Infra alert CI = HOST or K8s controller	CPU → HOST via SOS. Workload problems → Deployment (etc.) from <code>k8s-workload-name</code> .	SGC 1.15.0 has no Workload data source; name lookup is required.

Decision	Current rule	Why
One problem → many events	SGO emits one em_event per ImpactedEntity for the same <code>message_key</code> (ProblemID).	HOST + pod + workload rows can share one alert. That is connector fan-out, not Client vs Cluster.
Do not merge L2I Davis events	<code>dt.davis.is_merging_allowed=false</code> on log OpenPipeline	Prevents HOST-bundling a Master WARN with Lab3.
Create path	AMR + Flow action Create incident from Alert → <code>EvtMgmtIncidentHandler</code> → <code>EvtMgmtCustomIncidentPopulator</code> . Interim: BR <code>em-alert-create-log-incident</code> . AMR stays inactive until that Flow is published.	OOTB Create Incident subflow skips the populator.
Correlate incidents by SI + Class:Line	Short description <code>Critical log event – {sn-log-signature}</code>	Not <code>Event Log WARN</code> . Repeats bump urgency; Data Lookup sets priority.
TBAC partition	<code>sn-environment</code> + <code>sn-service-instance</code> + <code>sn-log-signature</code>	Stops cross-environment and cross-SI merge.

Specific observability platforms (e.g. Dynatrace or Elastic Stack) and specific ITSM platforms (e.g. ServiceNow or Remedy) will have specific requirements for implementing the Log to Incident observability process. However, there are the several design considerations that span the different platforms. These include:

- Which log messages are considered salient and should be turned into an incident?
- How to model the (service-side) application in the CMdb?
- How to model clients in the CMdb?
- What information must an event, whether from a client or a service, include to determine the CI to map to an incident?

8.3. OneAgent

OneAgent is Dynatrace’s single, unified data-collection agent that runs on each host. It performs full-stack monitoring of hosts, containers, and applications. For Log to Incident it either collects container stdout/stderr (Cluster Mode) or tails files under `/mnt/spark/client-logs/` (Client Mode) and sends each entry with contextual metadata to Dynatrace.

We need to tell OneAgent how to find the log stream and what contextual metadata to include with each log entry. That differs by mode.

8.3.1. Log Metadata

The log metadata is the key metadata fields that we need to extract from the log lines or the log context. This depends on what fields are required, the available log context, the available fields in a log line, Dynatrace’s ability to parse the log context, and the OpenPipeline’s ability to enrich or parse further.

OneAgent provides a limited set of default metadata fields (raw line, log level, log source).

OpenPipeline then stamps the **sn-*** dash keys used for ServiceNow bind. OpenPipeline **cannot** look up Dynatrace entities by name; it stamps **k8s-pod-name** / **k8s-workload-name** (and **sn-service-instance**) so ServiceNow can bind.

8.3.2. Cluster Mode

OneAgent on each Kubernetes node collects container stdout/stderr when DynaKube enables the Log module. Cluster Mode application logs do **not** use a custom log source path pattern for **/mnt/spark/logs**.

Listing 14. DynaKube logMonitoring (Container Output)

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
spec:
  # OneAgent Log module: autodiscover container stdout/stderr
  # (log.source=Container Output). Required for Cluster Mode Spark
  # app logs after log4j2-cluster Console appender.
  logMonitoring: {}
```

Repo: [observability/dynatrace/dynakube/dynakube.yaml.j2](https://github.com/observability/dynatrace/dynakube/dynakube.yaml.j2).

Table 19. OneAgent Container Output metadata fields (Cluster Mode)

Attribute	How it is set	Value
content	OneAgent (raw line)	Full Log4j line from stdout
loglevel	OneAgent (severity from line text)	WARN, ERROR, INFO, ...
log.source	OneAgent Log module	Container Output
k8s.namespace.name	OneAgent / Kubernetes enrichment	e.g. spark
k8s.pod.name	OneAgent / Kubernetes enrichment	e.g. spark-master-0 (pod metadata.name)
sn-service-instance	OpenPipeline (pod-name prefix)	Spark-Master / Spark-Worker / Spark-History-Server
Entity context	OneAgent Log module	May attach CAI / PGI / process context — unlike Client Mode HOST file-tail

8.3.3. Client Mode

The key requirements for Client Mode are that:

- We need to know the log level in order to determine which issues are critical or not.
- We need to mark the stream as Client Mode and capture the driver **instance** differentiator.
- We need a service instance bind key (**service_instance** on the custom source, mapped to **sn-service-instance**) for ServiceNow.

The custom log source below stamps the Client driver instance and **service_instance** from the path pattern (**SPARK_DRIVER_INSTANCE** is the directory under **/mnt/spark/client-logs/**).

Listing 15. Custom Log Source Configuration (Client Mode paths only)

```
{
  "schemaId": "builtin:logmonitoring.custom-log-source-settings",
  "scope": "environment",
  "value": {
    "enabled": true,
    "config-item-title": "{{ dt_l2i_standalone_custom_source_name }}",
    "custom-log-source": {
      "type": "LOG_PATH_PATTERN",
      "values-and-enrichment": [
        {
          "path": "/mnt/spark/client-logs/*/spark-app.log",
          "enrichment": [
            { "type": "attribute", "key": "log.path", "value": "${0}" },
            { "type": "attribute", "key": "log.driver.instance", "value": "${1}" },
            { "type": "attribute", "key": "service_instance", "value": "Spark-Client" }
          ]
        }
      ]
    }
  }
}
```

Given the above template, OneAgent will extract the following metadata fields and send them to Dynatrace.

Table 20. OneAgent and custom log source metadata fields (Client Mode)

Attribute	Value
content	Full Log4j line from the log file via OneAgent
loglevel	WARN, ERROR, INFO, etc., severity from OneAgent
log.source	Registered path under /mnt/spark/client-logs/... from OneAgent
log.path	Literal file path on the node set from log source \${0}
log.driver.instance	Driver directory under /mnt/spark/client-logs/ (par-a, PID, ...) set from log source \${1} (SPARK_DRIVER_INSTANCE)
service_instance	Spark-Client; the standalone pipeline maps this to sn-service-instance

8.3.4. Next Steps

The next is to send the logs to the Dynatrace SaaS platform. There they will go through a router that will route the logs to the appropriate OpenPipeline.

Once the logs are sent to Dynatrace in , they are routed to an OpenPipeline that processes the log entry. During the pipeline some log entries may be further processed then dropped, converted to problems, or stored in the Grail bucket. Those events converted to a problem are then forwarded to ServiceNow. Client file tails and Cluster Container Output use **two** pipelines (**standalone-short-log4j2-log-alerts** and **k8s-short-log4j2-log-alerts**) and share the alerting profile and webhook.

8.4. Log Routing

To route the logs to the designated OpenPipeline, we configure the Logs Routing object with **two**

entries.

Listing 16. OpenPipeline logs routing

```
{
  "schemaId": "builtin:openpipeline.logs.routing",
  "scope": "environment",
  "value": {
    "routingEntries": [
      {
        "enabled": true,
        "pipelineType": "custom",
        "pipelineId": "<standalone-short-log4j2-log-alerts object id>",
        "matcher": "matchesValue(log.source, \"/mnt/spark/client-logs/*\"),",
        "description": "Standalone short Log4j2 (Client file tail)"
      },
      {
        "enabled": true,
        "pipelineType": "custom",
        "pipelineId": "<k8s-short-log4j2-log-alerts object id>",
        "matcher": "matchesValue(log.source, \"Container Output\") AND matchesValue(k8s.namespace.name, \"spark\")",
        "description": "K8s short Log4j2 (Cluster Container Output)"
      }
    ]
  }
}
```

Ideally, all application log files would use a single pipeline. This is not possible given the way Kubernetes and standalone log files are processed. Then next best approach is would divide applications into classes based on their log files requirements. For example, self-developed Kubernetes application and third-party Java applications, where we could specify log file format standards for each class. For now, we will use two pipelines for just Spark Observability.

8.5. OpenPipeline processing

The Spark log processing OpenPipeline has three stages of processing:

1. **Processors** - where the log entries are processed and the metadata fields are extracted or defined
2. **Davis** - where the log entries are matched to a Davis processor to generate a Davis event that is ultimately converted to a problem and forwarded to ServiceNow.
3. **Grail** - where the Davis events are stored in the Grail bucket.

The templates for the two OpenPipelines share the same three stages. `customId` is `k8s-short-log4j2-log-alerts` or `standalone-short-log4j2-log-alerts`.

Listing 17. OpenPipeline skeleton (K8s and standalone)

```
{
  "schemaId": "builtin:openpipeline.logs.pipelines",
  "scope": "environment",
  "value": {
    "displayName": "K8s short Log4j2 log alerts",
    "customId": "k8s-short-log4j2-log-alerts",
    "routing": "routable",
    "groupRole": "memberPipeline",
    "processing": {}
  }
}
```

```

    "davis": {},
    "storage": {}
  }
}

```

8.5.1. Processors

Processors stamp **dash-key** **sn-*** fields for ServiceNow TBAC and parse the Log4j short **Class:Line** into **sn-log-signature**. OpenPipeline **cannot** look up entities by name.

1. **tag-pipeline-custom-id** — stamps **sn-pipeline** (the pipeline customId) and **sn-environment** on every record.
2. **extract-log4j-class-line** — parses **Class:Line** into **sn-log-class**, **sn-log-line**, **sn-log-signature**; sets **sn-impact** / **sn-urgency** (ERROR=2/2, WARN=3/3).

8.5.1.1. Cluster Mode

Container Output already supplies **k8s.pod.name**. Pipeline **k8s-short-log4j2-log-alerts** maps the pod-name prefix to **sn-service-instance** and adds dash aliases (**k8s-pod-name**, **k8s-workload-name**, ...).

Table 21. OpenPipeline entry fields (Cluster Mode)

Field Name	Cluster Mode
log.source	Container Output
k8s.pod.name	Kubernetes pod name (e.g. spark-master-0)
sn-service-instance	Spark-Master / Spark-Worker / Spark-History-Server from pod-name prefix
dt.source_entity	Often HOST or CAI. SN bind is sn-service-instance , not the pod CI

Cluster processors (template **k8s-short-log4j2-openpipeline.json.j2**): **tag-pipeline-custom-id**, **extract-log4j-class-line**, **derive-sn-service-instance-from-pod**, **alias-k8s-dash-keys**.

8.5.1.2. Client Mode

The custom log source already sets **service_instance** and **log.driver.instance**. Pipeline **standalone-short-log4j2-log-alerts** maps **service_instance** → **sn-service-instance**.

Table 22. OpenPipeline entry fields (Client Mode)

Field Name	Client Mode
log.driver.instance	Path segment under /mnt/spark/client-logs/ (SPARK_DRIVER_INSTANCE)
service_instance	Spark-Client (custom source)
sn-service-instance	Mapped from service_instance in the pipeline
dt.source_entity	HOST (OneAgent file tail). SN binds via sn-service-instance , not HOST

Client processors (template **standalone-short-log4j2-openpipeline.json.j2**): **tag-pipeline-custom-id**, **compose-log-instance**, **extract-log4j-class-line** (includes the **service_instance** → **sn-service-**

`instance` map).

8.5.2. Davis

Davis processing enables communication of critical issues to ServiceNow and provides opportunities to reduce noise or increase the strength of the signals sent to ServiceNow. Noise is reduced by eliding potential events that match an existing Davis event. Signal strength is increased by creating a Davis problem and grouping events that have the same root cause into a single Davis problem. We discuss how Davis processors work and then how we define the processors for handling Spark log entries.

8.5.2.1. Davis Processing

The Davis processors create Davis events or if a pipeline candidate's identity matches an existing Davis event, it will update the existing event. Davis defines identity as the tuple:

```
(event.name, event.type, dt.source_entity)
```

If this tuple is the same across the candidate in the pipeline and an existing Davis event, the existing event will be updated. If there was no match, a new Davis event will be created.

Once a Davis event is created, the Davis processor may create or update a Davis problem. A Davis processor is a rule that has a matcher. If the condition of the matcher is met, the Davis processor will create a Davis problem for an event or group the event into an existing Davis problem. If the condition of the matcher is not met, the Davis processor will drop the event from any further Davis processing, but not from any other Pipeline processing, like persisting the event to Grail.

If the matcher condition is met, the Davis processor will create a Davis problem or group the event into an existing Davis problem. Grouping is dependent on the problem's:

1. The matcher condition is met
2. The `dt.davis.is_merging_allowed` property is set to `true`
3. The event's timestamp is within the `dt.davis.event_timeout` window of the existing Davis problem
4. The event's `dt.source_entity` is the same as the existing problem's entity or topologically close in Smartscape as in process to host relationships or closely related services

The template in [Problem notification payload specification](#) is substituted by Dynatrace at send time. The body is `ProblemDetailsJSON` + `ImpactedEntities`. Attributes ServiceNow uses live in `rankedEvents[0].customProperties` (and as tokens in `event.description`). Use **dash keys** wherever TBAC or a Glide condition would treat `.` as a JSON path walk. Script Includes read dotted platform keys with bracket access (`customProperties['k8s.workload.name']`), then copy a dash alias.

Table 23. Payload attributes by context

Context	Who can stamp	Attributes SN uses	Alert CI lookup
K8s / standalone short Log4j2	OpenPipeline Davis (<code>is_merging_allowed=false</code>)	<code>sn-event-kind</code> , <code>sn-log-signature</code> , <code>sn-environment</code> , <code>sn-service-instance</code> , <code>sn-pipeline</code> , <code>sn-impact</code> , <code>sn-urgency</code> ; K8s also <code>k8s-pod-name</code> , <code>k8s-workload-name</code> , <code>k8s-workload-kind</code>	Service instance (enrich overwrites HOST/pod SOS)
Host CPU metric event	Metric event <code>eventTemplate.metadata</code>	<code>sn-event-kind=CPU_EVENT</code> , <code>sn-environment</code> , <code>sn-impact=2</code> , <code>sn-urgency=2</code>	HOST via SOS
OOTB K8s workload	DT metric dimensions (cannot add OpenPipeline metadata)	<code>k8s.workload.name</code> / kind / namespace; SN copies <code>k8s-workload-name</code>	Deployment (or kind table) by name . SGC 1.15.0 has no Workload SOS
OOTB K8s pod / CAI	DT	<code>k8s.pod.name</code> ; SN copies <code>k8s-pod-name</code>	Pod CI by name (not used for log incidents)
OOTB host <code>CPU_SATURATED</code>	None (no metadata schema)	<code>ProblemSeverity</code> only	HOST SOS; enrich fills <code>sn-*</code> from severity

8.5.2.1.1. Event Name

The `event.name` property should be a stable, human-readable event-class title and not contain any instance specific data of the event. Instance specific data would include data like the service instance name, pod name, or client instance name. Instance specific data should be placed in the `event.description` property.

Put the entity on `dt.source_entity`, the split key on `event.unique_identifier`, and the operator narrative on `event.description`. Current L2I (`Critical {loglevel} at {class}:{line}`) is a valid compromise because `class:line` is a bounded failure class when `dt.source_entity` is too coarse (HOST).

8.5.2.1.2. Source Entity

The `dt.source_entity` property is the entity that represents the source of the event.

8.5.2.1.3. Description

Instance specific data would include data like the service instance name, pod name, or client instance name. Instance specific data should be placed in the `event.description` property.

8.5.2.1.4. Unique Identifier

The `event.unique_identifier` property is a unique identifier for the event. The `event.unique_identifier` property is used to determine if the event is a duplicate of an existing event. The `event.unique_identifier` property is also used to determine if the event should be created as a new problem or grouped into an existing problem.

8.5.2.2. Tag Contract

ServiceNow-facing Dynatrace custom properties use the **sn-*** prefix with **all kebab-case** (dashes only — no underscores inside the name).

| Form | Verdict | |-----|-----| | **sn.service_instance** | Don't (dots — TBAC walks **.** as a JSON path) | | **sn-service-instance** | Prefer (all kebab) | | **sn_service_instance** | OK only if you standardize on all-snake — this lab does **not** | | **sn-service_instance** | Avoid (mixed - and _) |

Same rule for every other **sn-*** key (**sn-event-kind**, **sn-log-signature**, ...).

Table 24. Tag Contract

Attribute	Usage	Description
sn-event-kind	Mandatory	CRITICAL_LOG_EVENT or CPU_EVENT
sn-environment	Mandatory	TBAC partition; alerts from different environments must not merge
sn-service-instance	Mandatory	CSDM service instance display name in the Kubernetes label charset (Spark-Master , Spark-Client , ...)
sn-pipeline	Convenience	OpenPipeline customId
sn-log-signature	Mandatory	{sn-log-class}:{sn-log-line} — incident correlation key
sn-log-class / sn-log-line	Convenience	Parsed short Log4j2 call site
sn-impact / sn-urgency	Mandatory	ITSM 1–3 (ERROR 2/2, WARN 3/3)
k8s-pod-name / k8s-workload-name	K8s	Dash aliases of platform k8s.* keys for TBAC-safe lookup

event.name is **Critical {loglevel} at {sn-log-class}:{sn-log-line}**. **event.unique_identifier** is **{sn-environment}|{sn-service-instance}|...{sn-log-signature}**.

dt.davis.is_merging_allowed=false so Davis does not HOST-bundle unrelated signatures. A host metric event for CPU is a **separate** configuration: threshold 50%, **eventType=RESOURCE**, **sn-event-kind=CPU_EVENT**, **sn-impact=2** / **sn-urgency=2**. OOTB K8s anomaly settings (CPU ~40%, memory ~50%) **cannot** stamp **sn-***; ServiceNow enrich maps **ProblemSeverity**.

8.5.2.2.1. Cluster Mode

Table 25. Davis Properties (K8s short Log4j2)

Property Name	Description
event.type	CUSTOM_ALERT
sn-event-kind	CRITICAL_LOG_EVENT
event.name	Critical {loglevel} at {sn-log-class}:{sn-log-line}
event.unique_identifier	{sn-environment} {sn-service-instance} K8s Critical {loglevel} {sn-log-signature}
sn-service-instance	From pod-name prefix
dt.davis.is_merging_allowed	false

Property Name	Description
sn-pipeline	k8s-short-log4j2-log-alerts

Davis processor id: `k8s-short-log4j2-warn-error-davis` (matcher requires `k8s.pod.name` and `sn-log-class` / `sn-log-line`). Fail-visible: `k8s-short-log4j2-missing-pod-name-davis`.

8.5.2.2.2. Client Mode

Table 26. Davis Properties (standalone short Log4j2)

Property Name	Description
event.type	CUSTOM_ALERT
sn-event-kind	CRITICAL_LOG_EVENT
event.name	Critical {loglevel} at {sn-log-class}:{sn-log-line}
event.unique_identifier	{sn-environment} {sn-service-instance} Standalone Critical {loglevel} {sn-log-signature}
sn-service-instance	Mapped from custom-source <code>service_instance</code>
dt.davis.is_merging_allowed	false
sn-pipeline	standalone-short-log4j2-log-alerts

Davis processor id: `standalone-short-log4j2-warn-error-davis`. Fail-visible: `standalone-short-log4j2-missing-service-instance-davis`. CUSTOM_DEVICE “Spark Client” is **historical** — the bind is `sn-service-instance`, not a custom device entity.

Once a Davis problem is created the problem is forwarded to ServiceNow via the alerting profile in [Alerting profile for Spark Observability to ServiceNow](#).

8.5.3. Event Storage

After the problems are created, Dynatrace stores all of the **log records** in the `default_logs` bucket (storage processors below). The same bucket assignment applies to Client and Cluster Mode records that traverse this pipeline.

Listing 18. Spark OpenPipeline storage (default_logs bucket)

```
{
  "storage": {
    "processors": [
      {
        "id": "default-storage",
        "type": "bucketAssignment",
        "enabled": true,
        "description": "Store in default logs bucket",
        "matcher": "true",
        "bucketAssignment": { "bucketName": "default_logs" }
      }
    ]
  }
}
```

8.6. Problem Alerting

To emit a Davis problem to ServiceNow we have to create an alerting profile and associate it with the webhook endpoint in ServiceNow designed to receive the problem from Dynatrace. We also have to specify the webhook payload format. Client and Cluster Mode share the same alerting profile and brooks-lab webhook.

8.6.1. Alerting Profile

A Dynatrace alerting profile couples a management zone to a set of severity levels and event filters on the problems. Dynatrace will emit those problems that match levels and filters to ServiceNow.

The full alerting profile template is shown below.

Listing 19. Alerting profile for Spark Observability to ServiceNow

```
{
  "objectId": "...",
  "value": {
    "name": "Spark Observability - ServiceNow brooks-lab",
    "managementZone": "....",
    "severityRules": [
      { "severityLevel": "AVAILABILITY", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "ERRORS", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "PERFORMANCE", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "RESOURCE_CONTENTION", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "CUSTOM_ALERT", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" }
    ],
    "eventFilters": []
  }
}
```



OpenPipeline stamped Davis events as `event.type = CUSTOM_ALERT` for both WARN and ERROR log lines (K8s and standalone pipelines). That produces Dynatrace problems whose `ProblemSeverity` is `CUSTOM_ALERT`, which matches the alerting profile's `CUSTOM_ALERT` severity rule (and is what gets notified to ServiceNow).

OpenPipeline stamps Davis events as `event.type = CUSTOM_ALERT` for both WARN and ERROR log lines (K8s and standalone pipelines). That produces Dynatrace problems whose `ProblemSeverity` is `CUSTOM_ALERT`, which matches the alerting profile's `CUSTOM_ALERT` severity rule (and is what gets notified to ServiceNow).

Note that the alerting profile management zone is set to the Spark Observability management zone. This is important as the alerting profile will now filter out any problems that do not have a `dt.source_entity` in the Spark Observability management zone. This means that the impacted entities (hosts for OneAgent file tails and CPU, and Kubernetes workloads for Container Output) must be in the Spark Observability management zone, too.

8.6.1.1. Webhook Endpoint

The webhook endpoint template couples the alerting profile to the webhook endpoint in ServiceNow designed to receive the problem from Dynatrace. The full webhook endpoint template is shown below.

Listing 20. Problem notification webhook endpoint (SGC)

```
enabled: true
type: WEBHOOK
displayName: "ServiceNow brooks-lab - Spark Observability"
alertingProfile: "<Application Observability - ServiceNow OBJECT-ID"
webHookNotification:
  url: "https://optimizincdemo1.service-now.com/api/sn_em_connector/em/inbound_event?source=SGO-Dynatrace"
"
```

8.6.1.2. Cluster Mode

Ensure Kubernetes workloads (**CLOUD_APPLICATION spark-***) and related CAI instances are in MZ **Spark Observability** so Container Output problems notify SGO.

8.6.1.3. Client Mode

Impacted entities for Client Mode Davis problems are typically the OneAgent **HOST** that tailed **/mnt/spark/client-logs/**. That host must be in MZ **Spark Observability** for the shared alerting profile to forward the problem.

8.6.2. Problem notification payload template

8.6.2.1. Cluster Mode

Listing 21. Example Cluster Mode problem notification payload

```
{
  "ConnectionId": "a1b2c3d4e5f678901234567890abcdef0",
  "ProblemID": "-9876543210987654321",
  "ProblemTitle": "Critical ERROR at TaskSetManager:270",
  "ProblemSeverity": "CUSTOM_ALERT",
  "ProblemImpact": "INFRASTRUCTURE",
  "ProblemDetailsJSON": {
    "displayName": "Critical ERROR at TaskSetManager:270",
    "problemId": "-9876543210987654321",
    "rankedEvents": [
      {
        "entityId": "CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890",
        "entityName": "spark-master-0",
        "eventType": "CUSTOM_ALERT",
        "severityLevel": "CUSTOM_ALERT",
        "title": "Critical ERROR at TaskSetManager:270",
        "eventDescription": "K8s short Log4j2 critical ERROR at TaskSetManager:270 on spark-master-0 sn-event-kind=CRITICAL_LOG_EVENT sn-log-signature=TaskSetManager:270 sn-service-instance=Spark-Master: 2026-07-20 12:40:29 ERROR TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job"
      }
    ],
    "startTime": 1719502426000,
    "endTime": -1
  },
  "ProblemDetailsText": "K8s short Log4j2 critical ERROR at TaskSetManager:270 on spark-master-0 sn-event-kind=CRITICAL_LOG_EVENT sn-log-signature=TaskSetManager:270 sn-service-instance=Spark-Master:\n2026-07-20 12:40:29"
}
```

```

ERROR TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job\n\n1 impacted entity: spark-master-0",
"ImpactedEntities": [
  {
    "entityId": "CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890",
    "name": "spark-master-0",
    "type": "CLOUD_APPLICATION_INSTANCE"
  }
],
"ImpactedEntity": "spark-master-0",
"ProblemURL": "https://<tenant>.apps.dynatrace.com/ui/problems/<problem-id>",
"State": "OPEN",
"Tags": "Project:spark-observability",
"PID": "-9876543210987654321"
}

```

ServiceNow enrich binds via `sn-service-instance` → service instance (overwrites HOST/pod SOS).

8.6.2.2. Client Mode

Listing 22. Example Client Mode problem notification payload

```

{
  "ConnectionId": "a1b2c3d4e5f678901234567890abcdef0",
  "ProblemID": "-9876543210987654321",
  "ProblemTitle": "Critical ERROR at TaskSetManager:270",
  "ProblemSeverity": "CUSTOM_ALERT",
  "ProblemImpact": "INFRASTRUCTURE",
  "ProblemDetailsJSON": {
    "displayName": "Critical ERROR at TaskSetManager:270",
    "problemId": "-9876543210987654321",
    "rankedEvents": [
      {
        "entityId": "HOST-A1B2C3D4E5F67890",
        "entityName": "Lab3",
        "eventType": "CUSTOM_ALERT",
        "severityLevel": "CUSTOM_ALERT",
        "title": "Critical ERROR at TaskSetManager:270",
        "eventDescription": "Standalone short Log4j2 critical ERROR at TaskSetManager:270 in /mnt/spark/client-logs/par-a/spark-app.log sn-event-kind=CRITICAL_LOG_EVENT sn-log-signature=TaskSetManager:270 sn-service-instance=Spark-Client: 2026-07-20 12:40:29 ERROR TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job"
      }
    ],
    "startTime": 1719502426000,
    "endTime": -1
  },
  "ProblemDetailsText": "Standalone short Log4j2 critical ERROR at TaskSetManager:270 in /mnt/spark/client-logs/par-a/spark-app.log sn-event-kind=CRITICAL_LOG_EVENT sn-log-signature=TaskSetManager:270 sn-service-instance=Spark-Client:\n2026-07-20 12:40:29 ERROR TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job\n\n1 impacted entity: Lab3",
  "ImpactedEntities": [
    {
      "entityId": "HOST-A1B2C3D4E5F67890",
      "name": "Lab3",
      "type": "HOST"
    }
  ],
  "ImpactedEntity": "Lab3",
  "ProblemURL": "https://<tenant>.apps.dynatrace.com/ui/problems/<problem-id>",
  "State": "OPEN",
  "Tags": "Project:spark-observability",
  "PID": "-9876543210987654321"
}

```

Chapter 9. Stage 3 — ServiceNow

The following decisions are the contract between Dynatrace and ServiceNow Event Management for this implementation. They are the same rules listed under [Design considerations](#).

Table 27. Log to Incident design decisions (ServiceNow)

Decision	Current rule	Why
Dash keys, not dots	<code>sn-log-signature</code> , <code>sn-service-instance</code> , <code>sn-impact</code> , <code>k8s-workload-name</code>	TBAC / Glide conditions walk <code>additional_info</code> on <code>..</code> . A key named <code>k8s.workload.name</code> is treated as a JSON path, not one property.
Dynatrace owns initial impact/urgency	OpenPipeline and metric-event metadata stamp <code>sn-impact</code> / <code>sn-urgency</code> . OOTB Davis/K8s cannot; SN enrich maps <code>ProblemSeverity</code> .	Alert <code>severity</code> (EM 1–5) and incident <code>priority</code> (Data Lookup) are different numbers.
Log alert CI = service instance	<code>CRITICAL_LOG_EVENT</code> / <code>sn-service-instance</code> → <code>cmdb_ci</code> is the CSDM service instance. Enrich overwrites SGO HOST/pod SOS.	Operators assign from the SI, not Lab3. Incident CI is the same SI.
Infra alert CI = HOST or K8s controller	CPU → HOST via SOS. Workload problems → Deployment (etc.) from <code>k8s-workload-name</code> .	SGC 1.15.0 has no Workload data source; name lookup is required.
One problem → many events	SGO emits one em_event per ImpactedEntity for the same <code>message_key</code> (ProblemID).	HOST + pod + workload rows can share one alert. That is connector fan-out, not Client vs Cluster.
Do not merge L2I Davis events	<code>dt.davis.is_merging_allowed=false</code> on log OpenPipeline	Prevents HOST-bundling a Master WARN with Lab3.
Create path	AMR + Flow action Create incident from Alert → <code>EvtMgmtIncidentHandler</code> → <code>EvtMgmtCustomIncidentPopulator</code> . Interim: BR <code>em-alert-create-log-incident</code> . AMR stays inactive until that Flow is published.	OOTB Create Incident subflow skips the populator.
Correlate incidents by SI + Class:Line	Short description <code>Critical log event – {sn-log-signature}</code>	Not <code>Event Log WARN</code> . Repeats bump urgency; Data Lookup sets priority.
TBAC partition	<code>sn-environment</code> + <code>sn-service-instance</code> + <code>sn-log-signature</code>	Stops cross-environment and cross-SI merge.

ServiceNow Event Management is a three stage process. First, event data is received from external sources, like Dynatrace and an event is created. If the message key of the candidate event matches an existing event, the candidate event is considered a duplicate event, and it will get combined with the earlier event. Then, an event is either promoted to an alert or grouped into an existing alert. Alerts then optionally be promoted to an incident or grouped with an existing incident. Once an incident is created a support team is notified.

In a sense, creating the incident is the goal of event management. The incident represents a call to action to address a critical error condition. However, we need to minimize the noise in the

event management process to make it more useful and sustainable for the support team.

At each step in the process there is an opportunity to increase the signal to noise ratio. For example, in the first step, we can deduplicate events from the same source and time window. In the second step, we can group related events into a single alert. In the third step, we can group related alerts into a single incident.

9.1. Event Management

Once an event payload is received, the first step is to create an event object in ServiceNow. The table below shows how the webhook fields map to the event object fields.

Table 28. Webhook payload to `em_event` field mapping

em_event field	Population
source	SGO-Dynatrace (from URL)
message_key	ProblemID
description	ProblemDetailsText — must include full log line from OpenPipeline event.description
message	ProblemTitle
severity	Mapped from ProblemSeverity: <code>CUSTOM_ALERT</code> (log path) and <code>ERROR</code> → SGO numeric severity; <code>RESOURCE_CONTENTION</code> → Warning (4) (not Minor/3)
type	SGO copies the Dynatrace event enum (<code>CUSTOM_ALERT</code> , <code>RESOURCE_CONTENTION</code> , <code>CPU_SATURATED</code>). <code>sn-event-kind</code> is the L2I semantic type (<code>CRITICAL_LOG_EVENT</code> / <code>CPU_EVENT</code>)
node	After enrich: <code>sn-service-instance</code> stamp for logs; otherwise ImpactedEntity display name
resource	After enrich: <code>sn-service-instance:<stamp></code> for logs; <code>k8s-workload-name:<name></code> for workload problems
cmdb_ci	SOS first (<code>sys_object_source</code> , name=SGO-Dynatrace, id=entityId). Enrich then overwrites logs to the service instance; empty-CI workload → <code>k8s-workload-name</code>
cmdb_ci_type	Service instance for logs; <code>cmdb_ci_kubernetes_deployment</code> (or kind table) for <code>CLOUD_APPLICATION</code> ; <code>cmdb_ci_linux_server</code> for HOST
additional_info	JSON: ProblemURL, Tags, ImpactedEntities, full ProblemDetailsJSON (<code>rankedEvents[0].customProperties</code> holds <code>sn-*</code>)
time_of_event	Problem start time from ProblemDetailsJSON
correlation_id	PID when mapped

By design, SGO seeds `cmdb_ci` from Dynatrace `entityId` via `sys_object_source`. That first bind is often HOST (file tail) or a fan-out ImpactedEntity. Enrich then overwrites **log** events to the service instance (`sn-service-instance`). Infra events keep HOST SOS or bind a Kubernetes controller from `k8s-workload-name`.

9.2. CI binding procedure

1. Listener receives POST at `inbound_event?source=SGO-Dynatrace`.
2. SGO SOS: `sys_object_source` where `name=SGO-Dynatrace` and `id=<Dynatrace entityId>` → initial `cmdb_ci`.
3. If the event is a log (`sn-event-kind=CRITICAL_LOG_EVENT` or `sn-service-instance` present): **overwrite** `cmdb_ci` to the service instance (replaces HOST/pod SOS).
4. Else if `k8s-workload-name` is present: bind Deployment / StatefulSet / CronJob / Job by kind and name.
5. Else if `k8s-pod-name` is present **and** this is not a log event: bind `cmdb_ci_kubernetes_pod` by name.
6. Else leave the HOST SOS bind.

9.3. Entity type to CMDB class

Table 29. Dynatrace entity to CMDB class (after enrich)

Dynatrace / stamp	CMDB class	When to expect
<code>sn-service-instance</code> (log)	<code>cmdb_ci_service_discovered</code> / <code>cmdb_ci_service_by_tags</code>	CRITICAL_LOG_EVENT — overwrites SOS
<code>CLOUD_APPLICATION</code> / <code>k8s-workload-name</code>	<code>cmdb_ci_kubernetes_deployment</code> (or StatefulSet / CronJob / Job)	OOTB K8s resource; name lookup (SGC 1.15.0 has no Workload SOS)
<code>CLOUD_APPLICATION_INSTANCE</code> / <code>k8s-pod-name</code>	<code>cmdb_ci_kubernetes_pod</code>	Non-log CAI problems only
HOST	<code>cmdb_ci_linux_server</code>	Host CPU metric; SOS when no overwrite

9.4. Alert Management

SGO groups events that share a `message_key` (ProblemID) into one alert. If Dynatrace omitted a message key, Event Management would generate one from Source, Node, Type, Resource, and Metric Name — the ProblemID from the webhook should always populate `message_key`, so that fallback is unused here. Duplicate events for the same ProblemID join the existing alert (including SGO fan-out rows for HOST + CAI + workload).

Log alert text / short description is **Critical log event – Class:Line** (`sn-log-signature`). SOS still does the first CI bind; enrich then overwrites **log** alerts to the service instance. Do not use application-oriented rules that force Spark Client AS or a pod CI as the current design.

9.5. Description and alert text

Table 30. Required description and alert text fields

Field	Required content
em_event.description	OpenPipeline event.description (log line plus sn-* tokens); optional ProblemURL appended
em_alert.description	Same as em_event.description (copied by the alert rule)
em_alert.resource	Logs: sn-service-instance:<stamp> . Infra: k8s-workload-name:<name> or HOST
em_alert.cmdb_ci	Logs: service instance (after enrich). Infra: HOST or K8s controller

9.6. SGC and Event Management configuration

Table 31. SGC and Event Management configuration components

Component	Specification
sa_event_field_mapping	ProblemDetailsText → description; ProblemTitle → message; ProblemID → message_key; ERROR → Major (2); RESOURCE_CONTENTION → Warning (4)
EM event rule	Mark events Ready when source=SGO-Dynatrace and severity ≤ 3. Do not require cmdb_ci first — enrich sets SI on logs.
EM alert rule	Create alert from Ready events; copy cmdb_ci and description from event. Enrich on the alert overwrites HOST/pod SOS for logs.
ACL on cmdb_key_value	Roles that run incident scripts must read tags on workload CIs (see install documentation for tag ACL requirements)

9.7. Create incident bound to service instance

Incidents are created through **EvtMgmtIncidentHandler.createIncidentNoUpdate** → **EvtMgmtCustomIncidentPopulator** (the same path as Flow action **Create incident from Alert**). The interim business rule **em-alert-create-log-incident** calls that handler with **autoOpen=false** while the L2I AMR is inactive. Do **not** insert **incident** rows from a custom GlideRecord in a business rule — that skips the populator and leaves **em_alert.incident** unset.

The populator sets **incident.cmdb_ci** to the service instance, writes **em_alert.incident**, and correlates by SI + **Critical log event – {sn-log-signature}**.

9.8. Severity policy

Event Management **severity** (1–5) and ITSM **sn-impact** / **sn-urgency** (1–3) are different numbers. Log Davis events use **event.type=CUSTOM_ALERT**, so webhook **ProblemSeverity** is **CUSTOM_ALERT**. This instance also maps Dynatrace **ERROR** → EM **Major (2)** and **RESOURCE_CONTENTION** → EM **Warning (4)** (not Minor/3) when those severities appear. OpenPipeline stamps log ERROR as **sn-impact=2** / **sn-urgency=2** and WARN as 3/3 (independent of ProblemSeverity). The populator copies those onto the incident; Data Lookup computes **priority**. The create gate is **severity** ≤ 3, so Warning (4) CPU/K8s alerts get **sn-*** but no incident.

Table 32. Severity, impact, and priority

Field	Who sets it	This instance
<code>em_event.severity / em_alert.severity</code>	SGO from <code>ProblemSeverity</code>	ERROR / CUSTOM_ALERT → SGO numeric; RESOURCE_CONTENTION → Warning (4)
<code>sn-impact / sn-urgency</code>	OpenPipeline or enrich from <code>ProblemSeverity</code>	Log ERROR 2/2; WARN 3/3; host CPU 2/2
<code>incident.impact / incident.urgency</code>	<code>EvtMgmtCustomIncidentPopulator</code>	Copied from <code>sn-*</code> ; repeats bump urgency
<code>incident.priority</code>	ServiceNow Data Lookup (impact × urgency)	Not set in L2I code

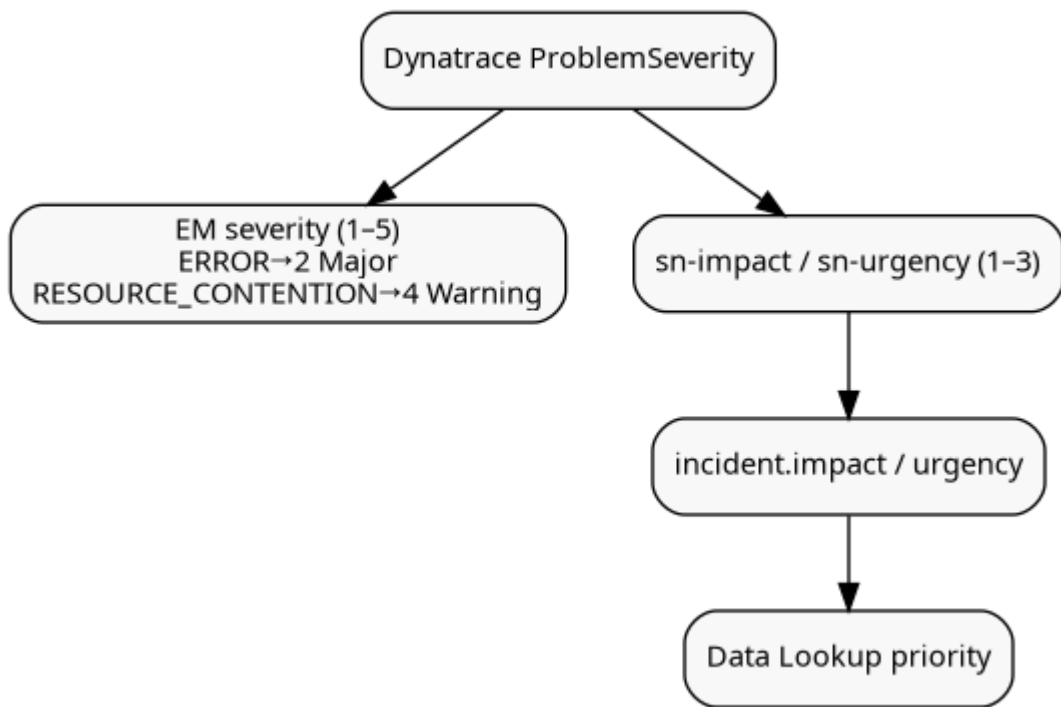


Figure 9. Priority stack

9.9. Layer model

Event, alert, and incident do **not** all share the same infrastructure CI. Logs bind the service instance at every layer. Infra alerts bind HOST or the Kubernetes **controller** (see [Kubernetes Architecture](#) — pods are the ephemeral band). The incident CI for logs is always the service instance.

Table 33. Event Management layer model and `cmdb_ci` classes

Layer	Table	<code>cmdb_ci</code> class
Event / Alert (log)	<code>em_event / em_alert</code>	Service instance (<code>cmdb_ci_service_discovered / cmdb_ci_service_by_tags</code>)
Event / Alert (host CPU)	<code>em_event / em_alert</code>	HOST (<code>cmdb_ci_linux_server</code>)

Layer	Table	cmdb_ci class
Event / Alert (K8s resource)	<code>em_event</code> / <code>em_alert</code>	Workload controller (<code>cmdb_ci_kubernetes_deployment</code> , StatefulSet, CronJob, ...)
Incident (log)	<code>incident</code>	Same service instance as the log alert

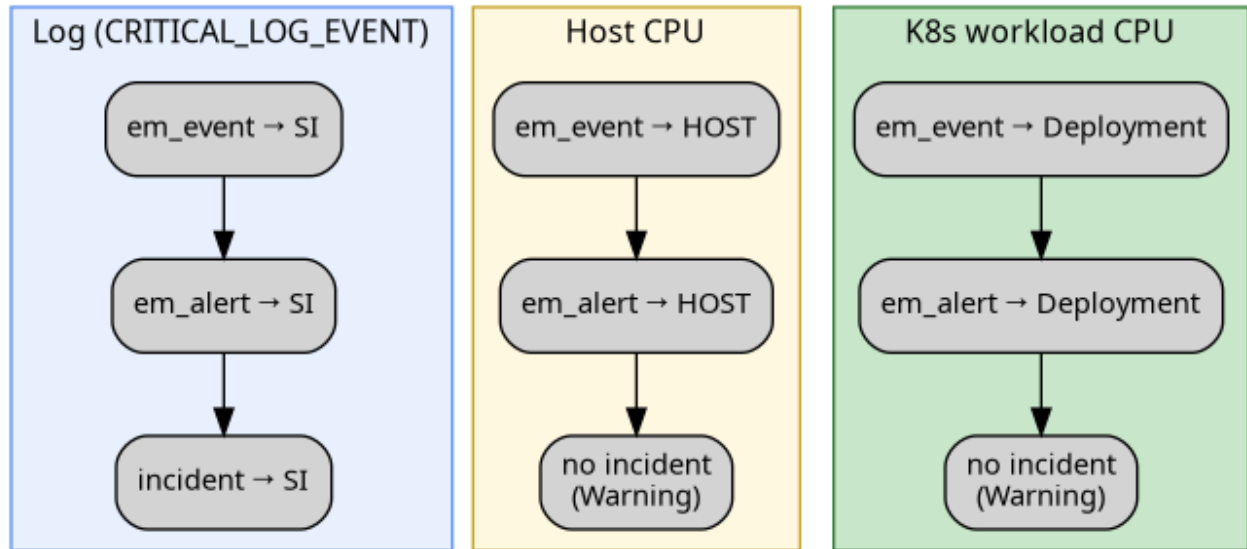


Figure 10. Event vs alert vs incident CI

9.10. Incident correlation

Incident correlation defines how enriched alerts map to `incident.cmdb_ci`. For logs, event, alert, and incident all use the **service instance**. Short description is **Critical log event – {sn-log-signature}**. Client and Cluster correlate the same way (SI + Class:Line). Repeats bump **urgency**; **priority** is ServiceNow Data Lookup (impact × urgency). The create gate remains `severity ≤ 3`, so Warning CPU/K8s resource alerts do not create incidents.

Table 34. Incident correlation patterns — alert CI vs service instance

Pattern	Alert CI	<code>incident.cmdb_ci</code> resolution
Log (Cluster or Client)	Service instance (<code>sn-service-instance</code>)	Same SI; correlate by SI + Critical log event – {sn-log-signature}
K8s workload resource	Deployment / StatefulSet / CronJob (<code>k8s-workload-name</code>)	No incident while EM severity is Warning
Host CPU	HOST via SOS	No incident while EM severity is Warning

9.10.1. Service-side

Cluster logs stamp `sn-service-instance` from the pod-name prefix (`spark-master*` → `Spark-Master`, and likewise Worker / History Server). Enrich overwrites SGO HOST/pod SOS so the alert CI is already the service instance. The populator copies that SI onto the incident and correlates by signature. `svc_ci_assoc` remains the fallback when the stamp is missing (controller or pod CI →

service instance). See [Kubernetes Architecture](#) for why the bind target is the workload/SI, not the pod.



Figure 11. Incident correlation — Cluster log

1. Enrich reads `sn-service-instance` and sets `em_event.cmdb_ci` / `em_alert.cmdb_ci` to that service instance.
2. Populator sets `incident.cmdb_ci` to the same SI and short description `Critical log event – {sn-log-signature}`.
3. An open incident with that SI + short description is reused; urgency is bumped.

9.10.2. Client-side

Client-mode driver logs are not a Kubernetes pod. The standalone pipeline maps custom-source `service_instance` → `sn-service-instance` (`Spark-Client`). Enrich and the populator then follow the same SI + Class:Line path as Cluster. Do **not** depend on `svc_ci_assoc` or a pod CI.



Figure 12. Incident correlation — Client log

9.11. CSDM identifier lookup

The **primary** log bind is the OpenPipeline stamp `sn-service-instance`, matched to the service instance `name` (Kubernetes label charset, for example `Spark-Master`). Path matching and `/client-logs/` are fallbacks when that stamp is missing.

CSDM `identifier` in `*.csdm.yaml` is the logical machine key. The table below shows which CMDB fields are reliable for lookup.

Table 35. CSDM identifier lookup fields on service instance

Purpose	Mechanism	Queryable field on <code>cmdb_ci_service_discovered</code>
Primary log bind	<code>sn-service-instance</code> stamp → SI display name	<code>name</code> (for example <code>Spark Master</code> / lookup via sanitized <code>Spark-Master</code>)
Fallback when the stamp is missing	Workload or pod CI → <code>svc_ci_assoc</code> membership	Tag-based Service Mapping on <code>servicenow.com/service-instance</code>
Manual / logical AS documentation	CSDM <code>identifier</code> in the spec	<code>identifier</code> column — often not populated on the instance

Purpose	Mechanism	Queryable field on <code>cldb_ci_service_discovered</code>
Client when <code>sn-service-instance</code> is absent	Standalone custom-source <code>service_instance</code> mapped to <code>sn-service-instance</code>	<code>name = Spark Client</code>

The Spark Client service instance **must** have `service_status=operational` so Event Management will assign to it.

9.12. Davis event identity vs problem merge

Davis identity for a log event is the tuple (`event.name`, `event.type`, `dt.source_entity`). Reports with the same tuple **refresh one Davis event**; `event.unique_identifier` alone does not split that identity. If merge is left on (`dt.davis.is_merging_allowed=true`), Davis can also fold nearby events into one **problem** when `dt.source_entity` is the same HOST or is topologically close in Smartscape — which is how a Master WARN and a Lab3 host problem historically became “Multiple infrastructure problems.”

Lab approach:

- OpenPipeline stamps dash keys: `sn-log-class`, `sn-log-line`, `sn-log-signature`, `sn-event-kind`, `sn-environment`, `sn-service-instance`.
- `event.name` is `Critical {loglevel} at {sn-log-class}:{sn-log-line}` so each call site is a distinct Davis event.
- `event.unique_identifier` is `{sn-environment}|{sn-service-instance}|...{sn-log-signature}` so the same Class:Line in a different environment or service instance does not share identity.
- `dt.davis.is_merging_allowed=false` so Davis does not HOST-bundle unrelated signatures.

SGO fan-out is not Client vs Cluster. One Dynatrace problem still produces **one em_event per ImpactedEntity** for the same `message_key` (ProblemID). A Master log problem can therefore arrive as HOST + CAI + `CLOUD_APPLICATION` rows. That is connector fan-out. Enrich binds the **log** rows to the service instance; leftover HOST rows on the same `message_key` are not Client-mode logs.

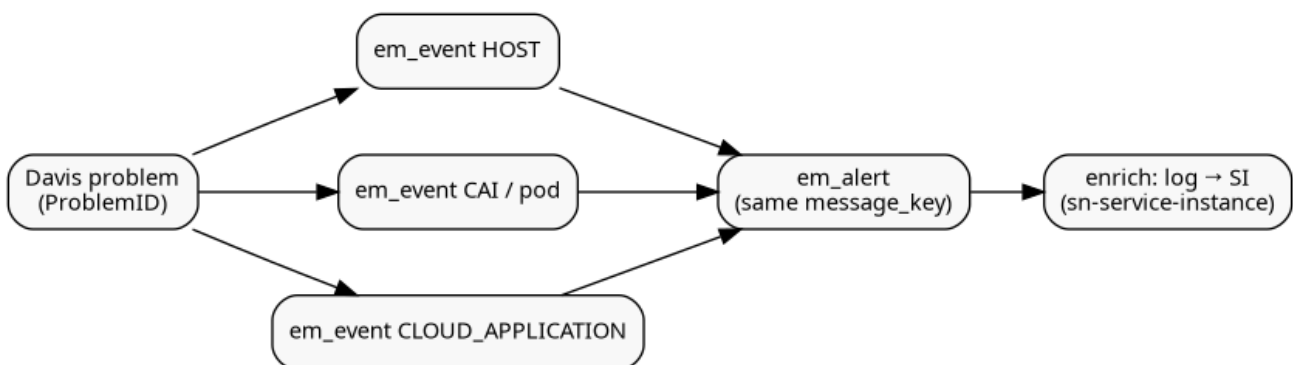


Figure 13. One Davis problem to N events to one alert

9.13. ServiceNow incident automation specification

Table 36. ServiceNow incident automation components

Component	Specification
Script Include: <code>ResolveApplicationService</code>	<code>enrichSgoRecord / applyLogServiceInstanceBinding / applyK8sTagCiBinding</code> . Log <code>cmb_ci</code> = service instance from <code>sn-service-instance</code> . Infra: HOST SOS or controller CI from <code>k8s-workload-name</code> .
Script Include: <code>EvtMgmtCustomIncidentPopulator</code>	Incident CI = service instance; correlate SI + <code>Critical log event - {sn-log-signature}</code> ; apply <code>sn-impact / sn-urgency</code>
Script Include: <code>L2IIncidentFromAlert</code>	Calls <code>EvtMgmtIncidentHandler.createIncidentNoUpdate(alert, false)</code> (<code>autoOpen=false</code>)
Business rule: <code>em-event-enrich-sgo</code>	Before insert/update on <code>em_event</code> ; order 100 ; <code>source=SGO-Dynatrace</code>
Business rule: <code>em-alert-enrich-sgo</code>	Before insert/update on <code>em_alert</code> ; order 5000 ; <code>source=SGO-Dynatrace</code>
Business rule: <code>em-alert-create-log-incident</code>	After insert/update on <code>em_alert</code> ; order 5020 ; <code>source=SGO-Dynatrace</code> <code>severity<=3;incidentIEMPTY</code>
TBAC	Definition L2I short Log4j2 SI + signature on dash keys <code>sn-environment</code> , <code>sn-service-instance</code> , <code>sn-log-signature</code>
AMR	L2I Create Incident CRITICAL_LOG_EVENT stays inactive until a published Flow uses action Create incident from Alert . Keep OOTB <code>SGO-Dynatrace</code> AMR inactive (that subflow skips the populator).
Create gate	<code>em_alert.severity <= 3</code> (Warning CPU/K8s resource alerts do not create incidents)

9.14. Business rule order and rationale

Business rules on `em_event` and `em_alert` run in **order** within the same **when** phase (**before** or **after**). Lower order runs first. Enrich must run before create so the 5020 shim sees the service instance and `sn-*` stamps. `autoOpen=false` on the handler while the L2I AMR is inactive.

Table 37. Business rule order on `em_event` and `em_alert`

Order	Rule	Why this position
100	<code>em-event-enrich-sgo</code> (before insert/update)	Stamp <code>sn-impact / sn-urgency</code> ; log → service instance; else K8s dash-key CI. Runs before EM Ready copies fields to the alert.
5000	<code>em-alert-enrich-sgo</code> (before insert/update)	Same on the alert — SGO may copy a HOST event onto the alert after the event enrich ran.
5020	<code>em-alert-create-log-incident</code> (after insert/update)	<code>EvtMgmtIncidentHandler.createIncidentNoUpdate(alert, false)</code> → populator. After-insert so <code>alert.sys_id</code> exists. Filter <code>source=SGO-Dynatrace</code> <code>severity<=3;incidentIEMPTY</code> .

9.15. Alert to incident linkage (`em_alert.incident`)

On `optimizincdemo1`, Event Management links an alert to its incident through `em_alert.incident` (reference to the incident record). That field populates the incident **Alerts** tab. Some instances expose the same reference as `em_alert.task`; this build uses **incident only** — do not reference `task`.

Creating an incident with `GlideRecord('incident').insert()` alone does **not** set this link. The handler writes `em_alert.incident`. `em_alert.task` may stay empty; do not require it.

Incident short description format: **Critical log event – {sn-log-signature}** (for example **Critical log event – TaskSetManager:270**). Correlation reuses an open incident with the same service instance and that short description; repeats bump **urgency**.

9.16. Problem and alert lifecycle (OPEN / RESOLVED)

Spark/Log4j does **not** emit explicit “clear” or “recovery” log lines for this use case. Closure is driven by **Dynatrace problem lifecycle**, not application log content:

1. OneAgent tails application log files on the **host node** (see [Log emission](#)).
2. OpenPipeline Davis turns qualifying **ERROR** / **WARN** lines into Davis events; repeated lines merge into one **problem** (`dt.davis.event_timeout: 15m`).
3. When matching log activity stops, Dynatrace **auto-closes** the problem after the timeout.
4. The brooks-lab problem notification has `notifyClosedProblems: true`, so Dynatrace POSTs a **RESOLVED** webhook with the same **ProblemID** as the OPEN post.
5. SGO maps `message_key = ProblemID`, so ServiceNow **updates the same `em_event`** (does not create a duplicate).
6. Event Management then transitions the linked **alert** (and optionally the **incident**) according to configured EM rules when the event state/resolution changes.

Verify RESOLVED handling: `em_event` where `source=SGO-Dynatrace` and `resolution_code` is set after problems clear; linked `em_alert.state` should move to **Closed** when EM auto-close rules are configured.

9.17. OpenPipeline and custom-source attributes (generic Log4j pattern)

Attributes stamped by the OpenPipeline templates (`k8s-short-log4j2-log-alerts` and `standalone-short-log4j2-log-alerts`). ServiceNow-facing keys use **all kebab** (`sn-service-instance`, `sn-event-kind`) — no dots and no underscores inside the `sn-*` name (see [Tag Contract](#)).

Table 38. OpenPipeline and custom log source attributes

Attribute	Source	Purpose
sn-log-class / sn-log-line	OpenPipeline parse of short Log4j2 <code>Class:Line</code>	Call-site identity
sn-log-signature	{sn-log-class}:{sn-log-line}	Incident short description and correlation key
sn-event-kind	Davis processor	<code>CRITICAL_LOG_EVENT</code> (logs) or <code>CPU_EVENT</code> (host metric)
sn-environment	OpenPipeline <code>tag-pipeline-custom-id</code>	TBAC partition (no cross-environment merge)
sn-service-instance	K8s: pod-name prefix → <code>Spark-Master / Spark-Worker / Spark-History-Server</code> . Standalone: custom-source <code>service_instance</code> mapped to this key	Primary log bind to the CSDM service instance
sn-pipeline	OpenPipeline <code>customId</code>	Which pipeline processed the line
sn-impact / sn-urgency	OpenPipeline (ERROR=2/2, WARN=3/3) or SN enrich from <code>ProblemSeverity</code>	ITSM 1–3; not EM <code>severity</code>
k8s-pod-name / k8s-workload-name / k8s-workload-kind	Dash aliases of platform <code>k8s.pod.name / k8s.workload.*</code>	TBAC-safe; workload CI lookup when the event is not a log
service_instance	Standalone OneAgent custom log source	Mapped to <code>sn-service-instance</code> in the standalone pipeline
event.type	Davis	Dynatrace enum (<code>CUSTOM_ALERT</code> , <code>RESOURCE_CONTENTION</code> , ...)
event.name	Davis	Critical {loglevel} at {sn-log-class}:{sn-log-line}
event.unique_identifier	Davis	{sn-environment} {sn-service-instance} ...{sn-log-signature}
dt.davis.is_merging_allowed	Davis	false for L2I logs

The Kubernetes label `servicenow.com/service-instance` is the CMDB/service-mapping contract on the **controller** and the pod template — not a Dynatrace tag. See [workload labels](#).



Cluster/Client Davis problems often keep OneAgent **HOST** as `dt.source_entity`. ServiceNow enrich overwrites log `cmdb_ci` to the service instance from `sn-service-instance`.

9.18. Event Management rules (manual configuration)

Configure under **Event Management** → **Rules**:

Table 39. Event Management rules for SGO-Dynatrace

Rule	Condition and action
Event rule	When: <code>em_event</code> insert/update; Filter: <code>source=SGO-Dynatrace^severity<=3</code> ; Action: set state to Ready . Do not require <code>cmdb_ciISNOTEMPTY</code> — log events become Ready with an SI <code>cmdb_ci</code> after enrich, or enrich sets it on the alert. Infra events may be HOST or a K8s controller CI.
Alert rule	When: Ready event; Filter: <code>source=SGO-Dynatrace</code> ; Action: create/update alert; copy <code>cmdb_ci</code> , <code>description</code> , <code>resource</code> , <code>node</code> , <code>message_key</code> from event. Enrich on the alert overwrites HOST/pod SOS for logs.

Scope Spark-specific business rules with `source=SGO-Dynatrace` so legacy `source=Dynatrace` events are unaffected during connector migration.

9.19. Script Includes

Deploy from `servicenow/integrations/incident/`. Global scope, **Active** = true.

Each Script Include below summarizes **purpose**, **inputs**, **outputs** / **side effects**, and how it fits the business rules. Source of truth is the `.si.js` file in the repository. `getCustomProperty` reads `additional_info` JSON with **bracket** keys (`customProperties['k8s.workload.name']`); never Glide-dot-walk dotted names.

9.19.1. ResolveApplicationService

Purpose: Enrich SGO-Dynatrace events/alerts and resolve the CSDM **service instance** for log binding.

Methods:

- `enrichSgoRecord(gr)` — Stamp `sn-impact` / `sn-urgency` (from OpenPipeline or mapped ProblemSeverity); copy dash aliases (`k8s-workload-name`, `k8s-pod-name`, ...). Then `applyLogServiceInstanceBinding` (logs) or `applyK8sTagCiBinding` (infra).
- `applyLogServiceInstanceBinding(gr)` — For `CRITICAL_LOG_EVENT` / `sn-service-instance`, overwrite `cmdb_ci` to the service instance; set `node` / `resource` to `sn-service-instance:<stamp>`.
- `applyK8sTagCiBinding(gr, keys)` — When `cmdb_ci` is empty: Deployment/StatefulSet/CronJob/Job by `k8s-workload-name` (and kind), else pod by `k8s-pod-name`.
- `extractSnImpact` / `extractSnUrgency` / `extractLogSignature` / `extractK8s*` — Read dash keys first, then dotted platform keys.
- `getCustomProperty(gr, keys)` — JSON.parse `additional_info` and ranked `customProperties`; try each key in order.
- `resolveServiceInstanceForIncident(gr)` — Primary: stamped `sn-service-instance` → SI name. Fallback: `svc_ci_assoc` from a workload/pod CI.

Inputs: `em_event` or `em_alert` GlideRecord (`source=SGO-Dynatrace`).

Outputs: Mutated `cmdb_ci`, `node`, `resource`, and `additional_info` stamps on the current row.

Source: `servicenow/integrations/incident/ResolveApplicationService.si.js`

9.19.2. EvtMgmtCustomIncidentPopulator

Purpose: Product hook called by `EvtMgmtIncidentHandler` (Flow action **Create incident from Alert**, or the interim create BR). Sets `incident.cmdb_ci` to the service instance; correlates an open incident with the same SI + short description **Critical log event – {sn-log-signature}**; copies `sn-impact` / `sn-urgency` (repeats bump urgency).

Inputs: `em_alert` and the in-flight `incident` task.

Outputs: `true` to continue insert; `false` to abort (correlated, or L2I log with no SI). Always writes `em_alert.incident` when a row is linked or created.

Source: `servicenow/integrations/incident/EvtMgmtCustomIncidentPopulator.si.js`

9.19.3. L2IIncidentFromAlert

Purpose: Handler helper. Calls `EvtMgmtIncidentHandler.createIncidentNoUpdate(alert, false)` so the populator runs without requiring an active AMR (`autoOpen=false`).

Inputs: `em_alert` GlideRecord.

Outputs: Incident `sys_id` when linked or created.

Source: `servicenow/integrations/incident/L2IIncidentFromAlert.si.js`

9.20. Business rules (prescriptive configuration)

Create under **System Definition** → **Business Rules**. Filter conditions must include `source=SG0-Dynatrace` so legacy connectors are not modified. Orders and when-clauses are in [Business rule order on `em\_event` and `em\_alert`](#).

Table 40. Prescriptive K8s log business rules

Order	When	Name	Filter	Responsibility
100	before	<code>em-event-enrich-sgo</code>	<code>source=SG0-Dynatrace</code>	Stamp <code>sn-impact/sn-urgency</code> ; log → service instance; else K8s dash-key CI
500	before	<code>em-alert-enrich-sgo</code>	<code>source=SG0-Dynatrace</code>	Same on the alert (SGO may copy a HOST event onto the alert)
5020	after	<code>em-alert-create-log-incident</code>	<code>source=SG0-Dynatrace</code> <code>severity<=3; incidentISEMPTY</code>	<code>EvtMgmtIncidentHandler.createIncidentNoUpdate(alert, false)</code>

9.21. Deployment Summary

Table 41. Deployment Summary

Kind	Configuration / template to implement
Dynatrace OpenPipeline (K8s logs)	Pipeline template equivalent to <code>k8s-short-log4j2-log-alerts</code> : parse short Log4j2; stamp <code>sn-event-kind</code> , <code>sn-log-signature</code> , <code>sn-environment</code> , <code>sn-service-instance</code> , <code>sn-impact/sn-urgency</code> ; dash aliases <code>k8s-pod-name</code> , <code>k8s-workload-name</code> , <code>k8s-workload-kind</code> ; <code>dt.davis.is_merging_allowed=false</code> ; <code>unique_identifier = env SI signature</code>
Dynatrace OpenPipeline (standalone / client logs)	Same contract as <code>standalone-short-log4j2-log-alerts</code> ; map custom-source <code>service_instance</code> → <code>sn-service-instance</code>
Dynatrace host metric event	Template: CPU above lab threshold (50%), <code>eventType=RESOURCE</code> , <code>sn-event-kind=CPU_EVENT</code> , <code>sn-impact=2</code> , <code>sn-urgency=2</code>
Dynatrace K8s anomaly settings	Lab-tuned percent rules (CPU ~40%, memory ~50%, short observation windows). OOTB schema cannot stamp <code>`sn-*</code>
Dynatrace problem notification	Payload template <code>ProblemDetailsJSON</code> + <code>ImpactedEntities</code> (attributes SN reads live in <code>rankedEvents[0].customProperties</code>)
K8s workload manifests	Controller labels <code>servicenow.com/service-instance</code> (and app labels) on Deployment/StatefulSet so CMDB <code>cmdb_key_value</code> lands on the controller CI
ServiceNow Script Includes	<code>ResolveApplicationService</code> (<code>enrichSgoRecord</code> , <code>applyLogServiceInstanceBinding</code> , <code>getCustomProperty</code> bracket reads); <code>EvtMgmtCustomIncidentPopulator</code> ; <code>L2IIncidentFromAlert</code>
ServiceNow business rules	<code>em-event-enrich-sgo</code> (before, 100); <code>em-alert-enrich-sgo</code> (before, 5000); <code>em-alert-create-log-incident</code> (after, 5020)
ServiceNow TBAC	Definition L2I short Log4j2 SI + signature on dash keys <code>sn-environment</code> , <code>sn-service-instance</code> , <code>sn-log-signature</code>
ServiceNow AMR	L2I Create Incident CRITICAL_LOG_EVENT inactive until a published Flow uses action Create incident from Alert . Keep OOTB <code>SGO-Dynatrace</code> AMR inactive

9.22. Example resolution

Three patterns share the same `ProblemID / message_key` path. They differ in which CI enrich leaves on the alert, and whether an incident is created (`severity<=3`).

Table 42. Example — log event (Spark Master)

Phase	Record / relationship
Problem	Davis <code>CUSTOM_ALERT</code> ; <code>sn-event-kind=CRITICAL_LOG_EVENT</code> ; <code>sn-service-instance=Spark-Master</code> ; <code>sn-log-signature=TaskSetManager:270</code>
Event	<code>em_event</code> ; SGO may bind HOST or pod first; enrich overwrites <code>cmdb_ci</code> → service instance Spark Master ; <code>node / resource = sn-service-instance:Spark-Master</code>
Alert	<code>em_alert</code> ; same service instance; short description Critical log event – TaskSetManager:270

Phase	Record / relationship
Incident	<code>incident.cmdb_ci</code> → the same Spark Master service instance (populator; correlate by SI + signature)

A Client log is the same pattern with `sn-service-instance=Spark-Client` and service instance **Spark Client**.

Table 43. Example — Kubernetes workload CPU

Phase	Record / relationship
Problem	OOTB K8s resource; <code>ImpactedEntities[0].type=CLOUD_APPLICATION</code> ; <code>k8s-workload-name=spark-worker-lab1</code>
Event / Alert	<code>cmdb_ci</code> → <code>cmdb_ci_kubernetes_deployment</code> spark-worker-lab1 (name lookup; SGC 1.15.0 has no Workload SOS)
Incident	None while EM severity is Warning (4) (<code>severity</code> ≤ 3 create gate)

Table 44. Example — host CPU

Phase	Record / relationship
Problem	Host metric event; <code>sn-event-kind=CPU_EVENT</code> ; <code>sn-impact=2</code> / <code>sn-urgency=2</code>
Event / Alert	<code>cmdb_ci</code> → HOST (lab1 or lab2) via SGO SOS
Incident	None while EM severity is Warning (4)

Chapter 10. Appendix A - Definitions

Table 45. Acronyms

Acronym	Full Form	Definition
	Affected entities	A broader set of entities associated with a problem and derived from an impacted entity. Typically, these are API entities where anomalies were detected.
	Impacted Entity	A monitored Smartscape entity that is experiencing degradation as indicated by the monitoring point and associated with a problem.
	Impacted Entities	Entities suffering degradation due to the problem (often the set shown as “impacted” in the UI and in notifications).
ITIL	Information Technology Infrastructure Library	A framework for IT service management that provides a common approach to managing IT services and align IT services with broader business goals.
	Root-cause entity	The entity that the Davis analysis engine identified as the root cause of the problem. It may be null.
SGO	Service Graph Observability	Out-of-the-box ServiceNow app/connector (SGO-Dynatrace) pulling Dynatrace topology, hosts, and service
SOS	Service Operations System	The enterprise event/CI processing and routing engine that maps ingested Dynatrace payloads into CSDM classes and applies custom ServiceNow enrichments.
TBAC	Tag-Based Alert Clustering	A ServiceNow Event Management feature that groups related em_alert records when they share configured tag/key values from the alert payload.

Chapter 11. Appendix B - Dynatrace

11.1. Severity levels

Dynatrace Davis event and problem **severityLevel** values (most to least severe), from [Event categories](#) and Events API v2 event-type metadata.

Table 46. Dynatrace Severity Levels

Severity Level	Description
MONITORING_UNAVAILABLE	Widespread monitoring interruption: most OneAgents lose connection with the Dynatrace server, so availability and performance visibility drop.
AVAILABILITY	High-severity incidents such as complete outage or unavailability of servers or processes.
ERROR	Increased error rates or other error-related incidents that interfere with regular operation.
PERFORMANCE	Slowdown: decreased performance of an operational service or application (less severe than error or availability).
RESOURCE_CONTENTION	Resource pressure such as CPU or memory saturation.
CUSTOM_ALERT	User-defined threshold / custom alert (any metric or OpenPipeline Davis event of this class).
INFO	Informational or warning-class events that do not open a problem on their own (deployments, configuration changes, administrative events, and WARNING event types).

11.2. Event Types

Full catalog of Dynatrace Davis **eventType** values and their associated **severityLevel**, from Events API v2 ([GET /api/v2/eventTypes](#)) on tenant **pdt20158** (118 types). Built-in detector types and OpenPipeline/ingest-assignable types are included. See also [Events API v2 — GET all event types](#) and [Event categories](#).

Table 47. Dynatrace Event Types and Severity Levels (pdt20158)

EventType	Severity Level	Display Name
APPLICATION_ERROR_RATE_INCREASED	ERROR	JavaScript error rate increase
APPLICATION_JS_FRAMEWORK_DETECTED	INFO	JavaScript framework change
APPLICATION_OVERLOAD_PREVENTION	INFO	Application overload prevention
APPLICATION_SLOWDOWN	PERFORMANCE	Application slowdown
APPLICATION_UNEXPECTED_HIGH_LOAD	RESOURCE_CONTENTION	Application high traffic
APPLICATION_UNEXPECTED_LOW_LOAD	AVAILABILITY	Application low traffic
AVAILABILITY_EVENT	AVAILABILITY	Availability event

EventType	Severity Level	Display Name
AWS_LAMBDA_HIGH_ERROR_RATE	ERROR	Lambda high error rate
CUSTOM_ALERT	CUSTOM_ALERT	Custom alert
CUSTOM_ANNOTATION	INFO	Custom annotation
CUSTOM_APPLICATION_ERROR_RATE_INCREASED	ERROR	Custom application error rate increase
CUSTOM_APPLICATION_OVERLOAD_PREVENTION	INFO	Custom application overload prevention
CUSTOM_APPLICATION_SLOWDOWN	PERFORMANCE	Custom application slowdown
CUSTOM_APPLICATION_UNEXPECTED_HIGH_LOAD	RESOURCE_CONTENTION	Custom application high load
CUSTOM_APPLICATION_UNEXPECTED_LOW_LOAD	AVAILABILITY	Custom application low load
CUSTOM_APP_CRASH_RATE_INCREASED	ERROR	Custom application crash rate increase
CUSTOM_CONFIGURATION	INFO	Configuration change
CUSTOM_DEPLOYMENT	INFO	Custom deployment
CUSTOM_INFO	INFO	Custom info
DATABASE_CONNECTION_FAILURE	ERROR	Failed database connects
DEPLOYMENT_CHANGED_CHANGE	INFO	Deployment changed
DEPLOYMENT_CHANGED_NEW	INFO	New deployment
DEPLOYMENT_CHANGED_REMOVED	INFO	Deployment removed
EC2_HIGH_CPU	RESOURCE_CONTENTION	AWS CPU saturation
ELB_HIGH_BACKEND_ERROR_RATE	ERROR	Load balancer backend error rate
ELB_HIGH_FRONTEND_ERROR_RATE	INFO	Load balancer frontend error rate
ELB_HIGH_UNHEALTHY_HOST_RATE	INFO	Load balancer unhealthy host rate
ERROR_EVENT	ERROR	Error event
ESXI_GUEST_CPU_LIMIT_REACHED	RESOURCE_CONTENTION	ESXi guest CPU saturation
ESXI_HOST_CONNECTION_FAILED	INFO	ESXi connection failed
ESXI_HOST_CONNECTION_LOST	INFO	ESXi connection lost
ESXI_HOST_CPU_SATURATION	RESOURCE_CONTENTION	ESXi host CPU saturation
ESXI_HOST_DATASTORE_LOW_DISK_SPACE	RESOURCE_CONTENTION	ESXi low datastore space
ESXI_HOST_DISK_QUEUE_SLOW	RESOURCE_CONTENTION	ESXi I/O commands queued
ESXI_HOST_DISK_SLOW	RESOURCE_CONTENTION	ESXi slow disk
ESXI_HOST_MAINTENANCE	INFO	ESXi maintenance
ESXI_HOST_MEMORY_SATURATION	RESOURCE_CONTENTION	ESXi host memory saturation
ESXI_HOST_NETWORK_PROBLEMS	ERROR	ESXi network problems

EventType	Severity Level	Display Name
ESXI_HOST_NO_CONNECTION	INFO	ESXi no connection
ESXI_HOST_OVERLOADED_STORAGE	RESOURCE_CONTENTION	ESXi overloaded storage
ESXI_HOST_SHUTDOWN	INFO	ESXi shutdown
ESXI_HOST_UP	INFO	ESXi start
ESXI_HOST_VM_MOTION_ARRIVED	INFO	VM migrated to this ESXi
ESXI_HOST_VM_MOTION_LEFT	INFO	VM migrated from this ESXi
ESXI_HOST_VM_STARTED	INFO	ESXi host VM started
ESXI_VM_DISCONNECTED	INFO	ESXi VM disconnected
ESXI_VM_IMPACT_HOST_CPU_SATURATION	RESOURCE_CONTENTION	ESXi VM impact host CPU saturation
ESXI_VM_IMPACT_HOST_MEMORY_SATURATION	RESOURCE_CONTENTION	ESXi VM impact host memory saturation
ESXI_VM_MOTION	INFO	ESXi VM motion
ESXI_VM_POWER_OFF	INFO	ESXi VM power off
ESXI_VM_SHUTDOWN	INFO	ESXi VM shutdown
ESXI_VM_START	INFO	ESXi VM start
EXTERNAL_SYNTHETIC_TEST_OUTAGE	AVAILABILITY	Third-party monitor outage
EXTERNAL_SYNTHETIC_TEST_SLOWDOWN	PERFORMANCE	Third-party monitor slowdown
HOST_LOG_AVAILABILITY	AVAILABILITY	Host availability log
HOST_LOG_ERROR	ERROR	Host error log
HOST_LOG_MATCHED	INFO	Host info log
HOST_LOG_PERFORMANCE	PERFORMANCE	Host performance log
HTTP_CHECK_GLOBAL_OUTAGE	AVAILABILITY	Http monitor global outage
HTTP_CHECK_LOCAL_OUTAGE	AVAILABILITY	Http monitor local outage
HTTP_CHECK_TEST_LOCATION_SLOWDOWN	PERFORMANCE	Http monitor slowdown
MARKED_FOR_TERMINATION	INFO	Marked for termination
MOBILE_APPLICATION_ERROR_RATE_INCREASED	ERROR	Mobile app error rate increase
MOBILE_APPLICATION_OVERLOAD_PREVENTION	INFO	Mobile app overload prevention
MOBILE_APPLICATION_SLOWDOWN	PERFORMANCE	Mobile app slowdown
MOBILE_APPLICATION_UNEXPECTED_HIGH_LOAD	RESOURCE_CONTENTION	Mobile app unexpected high usage
MOBILE_APPLICATION_UNEXPECTED_LOW_LOAD	AVAILABILITY	Mobile app unexpected low usage
MOBILE_APP_CRASH_RATE_INCREASED	ERROR	Mobile app crash rate increase
MONITORING_UNAVAILABLE	MONITORING_UNAVAILABLE	Monitoring unavailable

EventType	Severity Level	Display Name
MULTI_PROTOCOL_GLOBAL_OUTAGE	AVAILABILITY	Network availability monitor global outage
MULTI_PROTOCOL_LOCAL_OUTAGE	AVAILABILITY	Network availability monitor local outage
MULTI_PROTOCOL_LOCATION_SLOWDOWN	PERFORMANCE	Network availability monitor slowdown
OSI_DISK_LOW_INODES	RESOURCE_CONTENTION	Low number of inodes available
OSI_GRACEFULLY_SHUTDOWN	AVAILABILITY	Host gracefully shutdown
OSI_HIGH_CPU	RESOURCE_CONTENTION	CPU saturation
OSI_HIGH_MEMORY	RESOURCE_CONTENTION	Memory saturation
OSI_LOW_DISK_SPACE	RESOURCE_CONTENTION	Low disk space
OSI_NIC_DROPPED_PACKETS_HIGH	ERROR	High rate of dropped packets
OSI_NIC_ERRORS_HIGH	ERROR	High number of network errors
OSI_NIC_UTILIZATION_HIGH	RESOURCE_CONTENTION	High network utilization
OSI_SLOW_DISK	RESOURCE_CONTENTION	Slow disk
OSI_UNEXPECTEDLY_UNAVAILABLE	AVAILABILITY	Host or monitoring unavailable
PERFORMANCE_EVENT	PERFORMANCE	Performance event
PGI_CRASHED_INFO	INFO	Process crashed
PGI_LOG_AVAILABILITY	AVAILABILITY	Process availability log
PGI_LOG_ERROR	ERROR	Process error log
PGI_LOG_MATCHED	INFO	Process info log
PGI_LOG_PERFORMANCE	PERFORMANCE	Process performance log
PGI_MEMDUMP	INFO	Process memory dump
PGI_MYSQL_SLOW_QUERIES_RATE_HIGH	PERFORMANCE	High rate of slow queries
PGI_UNAVAILABLE	AVAILABILITY	Process unavailable
PG_LOW_INSTANCE_COUNT	AVAILABILITY	Process group low instance count
PROCESS_HIGH_GC_ACTIVITY	RESOURCE_CONTENTION	Long garbage-collection time
PROCESS_MEMORY_RESOURCE_EXHAUSTED	RESOURCE_CONTENTION	Memory resources exhausted
PROCESS_NA_HIGH_CONN_FAIL_RATE	ERROR	Connectivity problem
PROCESS_NA_HIGH_LOSS_RATE	ERROR	Network problem
PROCESS_RESTART	INFO	Process restart
PROCESS_THREADS_RESOURCE_EXHAUSTED	RESOURCE_CONTENTION	Threads resources exhausted
RDS_AZ_FAILOVER_COMPLETED	INFO	RDS failover completed
RDS_AZ_FAILOVER_STARTED	INFO	RDS failover started

EventType	Severity Level	Display Name
RDS_BACKUP_COMPLETED	INFO	RDS backup completed
RDS_BACKUP_STARTED	INFO	RDS backup started
RDS_HIGH_CPU	RESOURCE_CONTENTION	RDS CPU saturation
RDS_HIGH_LATENCY	RESOURCE_CONTENTION	High RDS latency
RDS_LOW_STORAGE_SPACE	RESOURCE_CONTENTION	Low RDS storage space
RDS_RESTART	INFO	RDS restart
RDS_RESTART_SEQUENCE	ERROR	RDS restart sequence
RESOURCE_CONTENTION_EVENT	RESOURCE_CONTENTION	Resource contention
SERVICE_ERROR_RATE_INCREASED	ERROR	Service failure rate increase
SERVICE_SLOWDOWN	PERFORMANCE	Service response time degradation
SERVICE_UNEXPECTED_HIGH_LOAD	RESOURCE_CONTENTION	Unexpected high service load
SERVICE_UNEXPECTED_LOW_LOAD	AVAILABILITY	Unexpected low service load
SYNTHETIC_GLOBAL_OUTAGE	AVAILABILITY	Browser monitor global outage
SYNTHETIC_LOCAL_OUTAGE	AVAILABILITY	Browser monitor local outage
SYNTHETIC_NODE_OUTAGE	AVAILABILITY	Synthetic ActiveGate outage
SYNTHETIC_PRIVATE_LOCATION_OUTAGE	AVAILABILITY	Synthetic private location outage
SYNTHETIC_TEST_LOCATION_SLOWDOWN	PERFORMANCE	Browser monitor slowdown
WARNING	INFO	Warning

11.2.1. Spelling quirks (important)

- Profile API uses **ERRORS** (plural) for the Error class.
- Webhook / problem field is usually **ERROR** (singular) for that class.
- For Custom alerts, profile and webhook both use **CUSTOM_ALERT** (same string).

11.2.2. How matching works

1. You set **event.type** (e.g. **CUSTOM_ALERT**) for OpenPipeline/custom events, or Dynatrace sets a built-in **eventType**.
2. Davis assigns the type's **severityLevel** (e.g. Custom → **CUSTOM_ALERT**).
3. Alerting profile matches that bucket via **severityLevel** (Error class uses profile **ERRORS**).
4. Problem notification substitutes **{ProblemSeverity}** from the problem.

Your Spark log path: **CUSTOM_ALERT** → problem **CUSTOM_ALERT** → profile **CUSTOM_ALERT** → webhook **ProblemSeverity: CUSTOM_ALERT**.

11.2.3. Not the same as numeric `event.severity` 1–5

There is also a newer ITIL-style `event.severity` (1–5). That is a different scale from classic profile buckets (`AVAILABILITY` / `ERRORS` / `CUSTOM_ALERT` / ...). Your OpenPipeline does **not** set that; classic problem severity for L2I log events comes from `event.type` → `CUSTOM_ALERT` class.